

**Plataforma Interactiva de Realidad Virtual
para la Visualización de Soluciones de Ecuaciones Diferenciales
Parciales
con Cómputo Remoto vía HTTP**

Universidad Nacional de Cuyo
Facultad de Ingeniería

Realidad Virtual (00318)

Proyecto Final de Cátedra

Profesores

Prof. Rosenstein, Javier

Prof. Tomba, Carlos

Alumnos

Escobar, Matías – 13328

Martín Duci, Ignacio – 13560

Agosto 2026

Índice

1. Introducción	5
2. Objetivos y Beneficiarios	6
2.1. Objetivo general	6
2.2. Objetivos específicos	6
2.3. Beneficiarios	6
3. Resolución de Ecuaciones Diferenciales	7
3.1. Fundamentos del método de diferencias finitas	7
3.2. Ecuación de onda 2D	7
3.2.1. Formulación matemática	7
3.2.2. Esquema leapfrog de segundo orden	8
3.2.3. Condición de estabilidad CFL	9
3.2.4. Condiciones iniciales implementadas	9
3.3. Ecuación de calor 2D	9
3.3.1. Formulación matemática	9
3.3.2. Esquema FTCS explícito	10
3.3.3. Condición de estabilidad	10
3.3.4. Condiciones iniciales y de contorno implementadas	10
4. Arquitectura General e Interconexión	11
5. Arquitectura del Servidor	12
5.1. Resumen de rutas	12
5.2. <code>main.go</code> — Punto de entrada y router	13
5.3. <code>auth.go</code> — Autenticación y limitación de tasa	13
5.4. <code>sync.go</code> — Estado global en memoria	15
5.5. <code>handlers.go</code> — Endpoints del panel web	21
5.6. <code>wave_api.go</code> — Endpoint dedicado a Unity	22
5.7. <code>heat_api.go</code> — Endpoint de calor dedicado a Unity	24
5.8. <code>admin.go</code> — Endpoints del panel docente	24
5.9. <code>gui.go</code> — Lanzador gráfico	27
5.10. <code>solver.go</code> — Implementación de los solvers	33
5.10.1. Solver de la ecuación de onda 2D	33
5.10.2. Solver de la ecuación de calor 2D	33
5.11. Consideraciones de eficiencia	37
6. Aplicación Móvil en Unity	38
6.1. Entorno de desarrollo y organización por escenas	38
6.2. Modos de visualización y sensores	39
6.3. Menú de inicio adaptable	40
6.4. Descubrimiento, registro y comunicación	41
6.5. Estructura de datos y reproducción	41
6.6. Escena de la ecuación de onda	41
6.6.1. Condición inicial personalizada	42

6.7. Escena de la ecuación de calor	42
6.8. Sincronización docente	42
7. Instalación y Reproducción del Sistema	44
7.1. Requisitos previos	44
7.2. Preparación y ejecución del servidor	45
7.2.1. Descarga de dependencias	45
7.2.2. Inicio mediante el lanzador gráfico	45
7.2.3. Verificación del panel docente	45
7.3. Apertura y verificación del proyecto Unity	45
7.4. Prueba previa dentro del editor	46
7.5. Generación e instalación del APK	46
7.6. Reproducción de una sesión completa	47
7.7. Diagnóstico de problemas frecuentes	48
7.8. Lista de verificación final	49
8. Pruebas	50
8.1. Servidor	50
8.2. Cliente	56
9. Conclusiones	58
10.Trabajo Futuro	59
10.1. Escalabilidad de la plataforma	59
10.2. Incorporación de nuevos solvers	59
10.3. Mejoras en la experiencia de realidad virtual	60
Referencias	61

Resumen

El presente trabajo describe el desarrollo de un sistema cliente-servidor orientado al entorno académico para el cálculo numérico y la visualización interactiva de ecuaciones diferenciales parciales (EDPs). El sistema fue concebido como una plataforma de exploración y aprendizaje que permite a los alumnos simular, probar e iterar sobre distintas configuraciones de EDPs — variando parámetros físicos, condiciones iniciales y dominios espaciales — para construir intuición sobre su comportamiento de forma activa. Como funcionalidad complementaria, el sistema incorpora un mecanismo de sincronización que permite al profesor propagar una configuración específica a todos los dispositivos conectados, facilitando la discusión guiada en clase a partir de un caso común.

El proyecto se estructura en dos componentes diferenciados. El primero corresponde al **servidor**, implementado en el lenguaje **Go** utilizando el framework **Gin** para la gestión de rutas HTTP. El servidor expone una API REST que permite el registro de usuarios, el cálculo de soluciones numéricas y el control de sincronización entre dispositivos. Se implementaron dos solvers de ecuaciones diferenciales parciales: la **ecuación de onda 2D**, resuelta mediante el esquema explícito *leapfrog* de segundo orden con condición CFL, y la **ecuación de calor 2D** sobre una placa cuadrada, resuelta mediante el esquema *FTCS* (*Forward Time Centered Space*) explícito. Ambos solvers operan sobre un dominio espacial discretizado con condiciones de contorno de Dirichlet, y emplean técnicas de *rolling buffer* y submuestreo temporal para mantener el uso de memoria acotado independientemente de la duración de la simulación. El servidor incluye además un panel web docente desarrollado en HTML, CSS y JavaScript, que permite visualizar las soluciones en una representación tridimensional renderizada por software mediante un rasterizador de polígonos implementado sobre el elemento **Canvas** de HTML5, con controles de órbita y un sistema de sincronización de parámetros hacia los alumnos.

El segundo componente corresponde a la **aplicación móvil**, desarrollada en **Unity** y compilada para Android. La aplicación descubre automáticamente el servidor dentro de la red local, registra al estudiante y consume las soluciones numéricas completas sin ejecutar el método de diferencias finitas en el teléfono. Los resultados se presentan sobre placas tridimensionales mediante color, desplazamiento, grilla y nodos visibles. La cámara utiliza los sensores de orientación del dispositivo y ofrece tanto una experiencia monoscópica a pantalla completa como un modo estereoscópico opcional para Google Cardboard. El teléfono fue seleccionado como dispositivo IHM porque constituye un recurso disponible para la totalidad de los alumnos de la cátedra; por este motivo, el visor no es necesario para desarrollar la actividad. El alumno puede modificar parámetros, alternar entre onda y calor, desplazar la placa, explorar modos naturales y editar una condición inicial seleccionando nodos. El cliente incorpora además una interfaz de inicio adaptable a distintas pantallas Android y un coordinador que aplica las configuraciones sincronizadas por el docente.

Palabras clave: ecuaciones diferenciales parciales, diferencias finitas, Unity, Android, realidad virtual, Google Cardboard, arquitectura cliente-servidor.

1. Introducción

Las ecuaciones diferenciales parciales (EDPs) constituyen una de las herramientas matemáticas fundamentales en ingeniería y ciencias aplicadas. Modelan fenómenos físicos tan diversos como la propagación de ondas, la difusión de calor, el flujo de fluidos y la deformación de estructuras. Sin embargo, su comprensión intuitiva representa un desafío significativo para los estudiantes, en parte debido a que los métodos de resolución analítica se limitan a casos muy particulares y las soluciones numéricas suelen presentarse de forma estática en hojas de cálculo o gráficas bidimensionales. Los métodos de diferencias finitas permiten aproximar estas soluciones mediante una discretización espacial y temporal del problema continuo [1].

El presente proyecto propone un enfoque alternativo: una plataforma interactiva que permite al usuario simular el comportamiento de EDPs durante la ejecución, variar libremente sus parámetros y condiciones iniciales, y observar las soluciones mediante una visualización tridimensional controlada por la orientación del teléfono. El modo plano concentra la interacción táctil, mientras que el modo Google Cardboard ofrece renderizado estereoscópico y seguimiento rotacional de tres grados de libertad. El objetivo no es reemplazar la teoría sino complementarla, ofreciendo un entorno donde la experimentación directa genere intuición física sobre fenómenos que de otro modo resultan abstractos.

El sistema se compone de dos componentes principales que se comunican mediante una arquitectura cliente-servidor HTTP. El servidor, desarrollado en Go, se encarga del cálculo numérico de las soluciones y de gestionar múltiples clientes simultáneos. El cliente, una aplicación móvil desarrollada en Unity, consume las soluciones del servidor y las presenta en dos modalidades: una vista monoscópica interactiva y una vista estereoscópica opcional orientada a Google Cardboard.

Como funcionalidad adicional orientada al uso en aula, el sistema incorpora un mecanismo de sincronización que permite al profesor propagar una configuración específica a todos los dispositivos conectados, habilitando la discusión guiada sobre un caso común.

2. Objetivos y Beneficiarios

2.1. Objetivo general

Desarrollar una plataforma cliente-servidor para la simulación interactiva de ecuaciones diferenciales parciales, con visualización tridimensional y un modo de realidad virtual opcional, que facilite el aprendizaje activo mediante la exploración paramétrica de soluciones numéricas en tiempo real.

2.2. Objetivos específicos

- Implementar solvers numéricos estables y eficientes para la ecuación de onda 2D y la ecuación de calor 2D sobre dominios cuadrados.
- Diseñar una API HTTP que exponga los resultados de la simulación a clientes heterogéneos (app móvil, panel web) de forma eficiente y con control de acceso por roles.
- Desarrollar una aplicación móvil en Unity que presente las soluciones numéricas en una escena tridimensional, permita la interacción táctil en modo plano y ofrezca visualización estereoscópica opcional para Google Cardboard.
- Proveer un panel web docente que permita visualizar, controlar y sincronizar simulaciones hacia múltiples usuarios conectados.
- Garantizar la estabilidad numérica de los esquemas implementados mediante el cumplimiento de las condiciones de estabilidad correspondientes a cada método.

2.3. Beneficiarios

La plataforma está orientada principalmente a **estudiantes de carreras de ingeniería y ciencias exactas** que cursen asignaturas donde las EDPs tienen un rol central: mecánica de medios continuos, transferencia de calor, electromagnetismo, mecánica de fluidos, entre otras. La posibilidad de explorar libremente parámetros y condiciones iniciales convierte al sistema en una herramienta de estudio autónomo que complementa la clase magistral.

En segundo lugar, la plataforma beneficia a los **docentes**, que pueden utilizarla como recurso didáctico durante la clase para ilustrar conceptos con simulaciones en vivo, o proponer a los alumnos escenarios específicos mediante la funcionalidad de sincronización.

3. Resolución de Ecuaciones Diferenciales

3.1. Fundamentos del método de diferencias finitas

Las EDPs implementadas en este proyecto se resuelven mediante el **método de diferencias finitas** (MDF), que consiste en discretizar el dominio continuo en una grilla de puntos y aproximar las derivadas parciales mediante cocientes de diferencias entre valores en nodos vecinos.

Dado un dominio espacial cuadrado $[0, L] \times [0, L]$ con n subdivisiones, el paso espacial es $\Delta x = \Delta y = L/n$. Los nodos interiores son los puntos $(i\Delta x, j\Delta y)$ con $1 \leq i, j \leq n-2$. Las condiciones de contorno de Dirichlet se imponen directamente sobre los nodos del borde, fijando su valor para todo tiempo.

La aproximación por diferencias centradas de segundo orden para cada derivada segunda espacial es:

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2}, \quad \frac{\partial^2 u}{\partial y^2} \approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\Delta y^2} \quad (1)$$

El laplaciano $\nabla^2 u = \partial^2 u / \partial x^2 + \partial^2 u / \partial y^2$ se obtiene sumando ambas expresiones. Como el dominio es una grilla cuadrada uniforme ($\Delta x = \Delta y = L/n$), los dos denominadores son iguales y los términos en $u_{i,j}$ se suman: $-2u_{i,j}/\Delta x^2 - 2u_{i,j}/\Delta y^2 = -4u_{i,j}/\Delta x^2$. El resultado es el **laplaciano discreto de 5 puntos**:

$$\nabla^2 u_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{\Delta x^2} \quad (2)$$

Esta expresión es la base de ambos esquemas numéricos implementados.

3.2. Ecuación de onda 2D

3.2.1. Formulación matemática

La ecuación de onda 2D base describe la evolución temporal de una membrana vibrante:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \quad (3)$$

La implementación extiende este modelo con amortiguamiento viscoso opcional y excitación forzada armónica:

$$\frac{\partial^2 u}{\partial t^2} + \gamma \frac{\partial u}{\partial t} = c^2 \nabla^2 u + A \sin(2\pi f t) \phi(x, y) \quad (4)$$

donde $u(x, y, t)$ es el desplazamiento de la membrana, c la velocidad de propagación, $\gamma \geq 0$ el coeficiente de amortiguamiento viscoso, f la frecuencia de excitación forzada [Hz] y $\phi(x, y)$ la forma espacial de la fuente. Se imponen condiciones de contorno de Dirichlet homogéneas ($u = 0$ en todo el borde) y velocidad inicial nula ($\partial u / \partial t|_{t=0} = 0$).

3.2.2. Esquema leapfrog de segundo orden

Se utiliza el esquema explícito *leapfrog* centrado en tiempo y espacio, que aproxima la derivada segunda temporal:

$$\frac{\partial^2 u}{\partial t^2} \approx \frac{u_{i,j}^{k+1} - 2u_{i,j}^k + u_{i,j}^{k-1}}{\Delta t^2} \quad (5)$$

El laplaciano del miembro derecho se aproxima aplicando la expresión (2) definida en la sección anterior:

$$c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = c^2 \nabla^2 u \approx c^2 \frac{u_{i+1,j}^k + u_{i-1,j}^k + u_{i,j+1}^k + u_{i,j-1}^k - 4u_{i,j}^k}{\Delta x^2} \quad (6)$$

Sustituyendo esta expresión junto con la aproximación temporal en (3) se igualan ambos miembros:

$$\frac{u_{i,j}^{k+1} - 2u_{i,j}^k + u_{i,j}^{k-1}}{\Delta t^2} = c^2 \frac{u_{i+1,j}^k + u_{i-1,j}^k + u_{i,j+1}^k + u_{i,j-1}^k - 4u_{i,j}^k}{\Delta x^2} \quad (7)$$

Multiplicando ambos miembros por Δt^2 y definiendo $r^2 = (c \Delta t / \Delta x)^2$:

$$u_{i,j}^{k+1} - 2u_{i,j}^k + u_{i,j}^{k-1} = r^2 (u_{i+1,j}^k + u_{i-1,j}^k + u_{i,j+1}^k + u_{i,j-1}^k - 4u_{i,j}^k) \quad (8)$$

Despejando $u_{i,j}^{k+1}$:

$$u_{i,j}^{k+1} = 2u_{i,j}^k - u_{i,j}^{k-1} + r^2 (u_{i+1,j}^k + u_{i-1,j}^k + u_{i,j+1}^k + u_{i,j-1}^k - 4u_{i,j}^k) \quad (9)$$

donde $r = c \Delta t / \Delta x$ es el **número de Courant**: mide cuántas celdas espaciales recorre la perturbación en un paso temporal. Un valor $r > 1/\sqrt{2}$ significa que la perturbación “salta” más rápido de lo que el esquema puede representar, provocando inestabilidad numérica.

Cuando se activa amortiguamiento ($\gamma > 0$) y/o excitación forzada, el esquema se generaliza a:

$$u_{i,j}^{k+1} = \frac{1}{1 + \gamma \Delta t} \left[2u_{i,j}^k - (1 - \gamma \Delta t) u_{i,j}^{k-1} + r^2 (u_{i+1,j}^k + u_{i-1,j}^k + u_{i,j+1}^k + u_{i,j-1}^k - 4u_{i,j}^k) + A_{\text{exc}} \sin(2\pi f t_k) \phi_{i,j} \right] \quad (10)$$

donde $A_{\text{exc}} = 0,15 \Delta t^2$ escala la amplitud de la fuente y $\phi_{i,j}$ es su forma espacial, coincidente con el modo de vibración seleccionado:

$$\phi_{i,j} = \sin\left(\frac{m_1 \pi x_i}{L}\right) \sin\left(\frac{m_2 \pi y_j}{L}\right)$$

Para $\gamma = 0$ y sin excitación el esquema se reduce a (9).

Para el primer paso temporal, donde no existe u^{k-1} , se aplica la condición de velocidad inicial nula para obtener un arranque de primer orden:

$$u_{i,j}^1 = u_{i,j}^0 + \frac{r^2}{2} (u_{i+1,j}^0 + u_{i-1,j}^0 + u_{i,j+1}^0 + u_{i,j-1}^0 - 4u_{i,j}^0) \quad (11)$$

3.2.3. Condición de estabilidad CFL

Para que el esquema leapfrog sea estable en 2D, el número de Courant debe satisfacer:

$$r = \frac{c \Delta t}{\Delta x} \leq \frac{1}{\sqrt{2}} \quad (12)$$

En la implementación se usa $r = 0,9/\sqrt{2}$ para mantener un margen de seguridad. El paso temporal resultante es $\Delta t = r \Delta x / c$, y el número de pasos temporales necesario para simular T segundos es $n_t = \lceil T / \Delta t \rceil$.

3.2.4. Condiciones iniciales implementadas

Se ofrecen tres configuraciones de condición inicial $u(x, y, 0)$:

- **Seno:** modo fundamental de vibración de la membrana cuadrada:

$$u_0 = \sin\left(\frac{\pi x}{L}\right) \sin\left(\frac{\pi y}{L}\right)$$

- **Gaussiana:** pulso centrado que se propaga como ondas circulares:

$$u_0 = \exp\left[-\left(\frac{x - L/2}{L/5}\right)^2 - \left(\frac{y - L/2}{L/5}\right)^2\right]$$

- **Triangular:** pirámide lineal centrada:

$$u_0 = (1 - |2x/L - 1|)(1 - |2y/L - 1|)$$

- **Modo** (m_1, m_2) : modo de vibración de orden superior, donde m_1 es la cantidad de semiondas a lo largo del eje x y m_2 a lo largo del eje y , codificado como la cadena "modo: $m_1:m_2$ " con $m_1, m_2 \in \{1, \dots, 5\}$:

$$u_0 = \sin\left(\frac{m_1 \pi x}{L}\right) \sin\left(\frac{m_2 \pi y}{L}\right)$$

- **Personalizada:** la app Unity puede enviar una grilla dibujada por el usuario (hasta 128×128 valores en el rango $[-2, 2]$); el servidor la interpola bilinealmente a la resolución numérica interna antes de iniciar el solver.

3.3. Ecuación de calor 2D

3.3.1. Formulación matemática

La ecuación de calor 2D describe la difusión de temperatura sobre una placa cuadrada:

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \quad (13)$$

donde $u(x, y, t)$ es la temperatura normalizada y α es la difusividad térmica del material.

3.3.2. Esquema FTCS explícito

Se utiliza el esquema *Forward Time Centered Space* (FTCS), que aproxima la derivada temporal mediante diferencias hacia adelante de primer orden:

$$\frac{\partial u}{\partial t} \approx \frac{u_{i,j}^{k+1} - u_{i,j}^k}{\Delta t} \quad (14)$$

Sustituyendo ambas aproximaciones en la ecuación de calor (13):

$$\frac{u_{i,j}^{k+1} - u_{i,j}^k}{\Delta t} = \alpha \frac{u_{i+1,j}^k + u_{i-1,j}^k + u_{i,j+1}^k + u_{i,j-1}^k - 4u_{i,j}^k}{\Delta x^2} \quad (15)$$

Multiplicando ambos miembros por Δt y definiendo $r_\alpha = \alpha \Delta t / \Delta x^2$, y despejando $u_{i,j}^{k+1}$:

$$u_{i,j}^{k+1} = u_{i,j}^k + r_\alpha (u_{i+1,j}^k + u_{i-1,j}^k + u_{i,j+1}^k + u_{i,j-1}^k - 4u_{i,j}^k) \quad (16)$$

donde $r_\alpha = \alpha \Delta t / \Delta x^2$ es el número de difusión.

A diferencia del esquema leapfrog, el FTCS solo requiere el instante anterior para avanzar en tiempo, lo que simplifica la implementación al necesitar únicamente dos grillas en lugar de tres.

3.3.3. Condición de estabilidad

Para que el esquema FTCS sea estable en 2D, el número de difusión debe satisfacer:

$$r_\alpha = \frac{\alpha \Delta t}{\Delta x^2} \leq \frac{1}{4} \quad (17)$$

En la implementación se usa $r_\alpha = 0,20$ como margen de seguridad respecto al límite teórico de 0,25.

3.3.4. Condiciones iniciales y de contorno implementadas

Se ofrecen tres configuraciones:

- **Gaussiana:** pulso de temperatura centrado. Los cuatro bordes se fijan en $u = 0$ (sumideros fríos). El calor se disipa hacia los bordes y la temperatura interior decae hasta equilibrarse en cero.
- **Bordes calientes:** los cuatro bordes se fijan en $u = 1$ (fuentes de calor constantes) y el interior comienza en $u = 0$. El calor penetra desde las paredes hacia el centro hasta alcanzar el estado estacionario $u = 1$ en todo el dominio.
- **Placa caliente:** el interior comienza en $u = 1$ y los cuatro bordes se fijan en $u = 0$ (sumideros fríos). El calor fluye desde el interior hacia los bordes hasta alcanzar el estado estacionario $u = 0$ en todo el dominio.

En ambos casos las condiciones de contorno se reimponen explícitamente después de cada paso temporal para mantener el valor fijo en el borde independientemente de la difusión numérica.

4. Arquitectura General e Interconexión

La Figura 1 resume la organización completa del sistema, los algoritmos numéricos ejecutados en Go y los flujos de información entre el panel docente, el servidor y las aplicaciones Android. Las flechas continuas distinguen el sentido de cada petición y respuesta; la flecha discontinua representa la propagación de la configuración sincronizada por el docente.

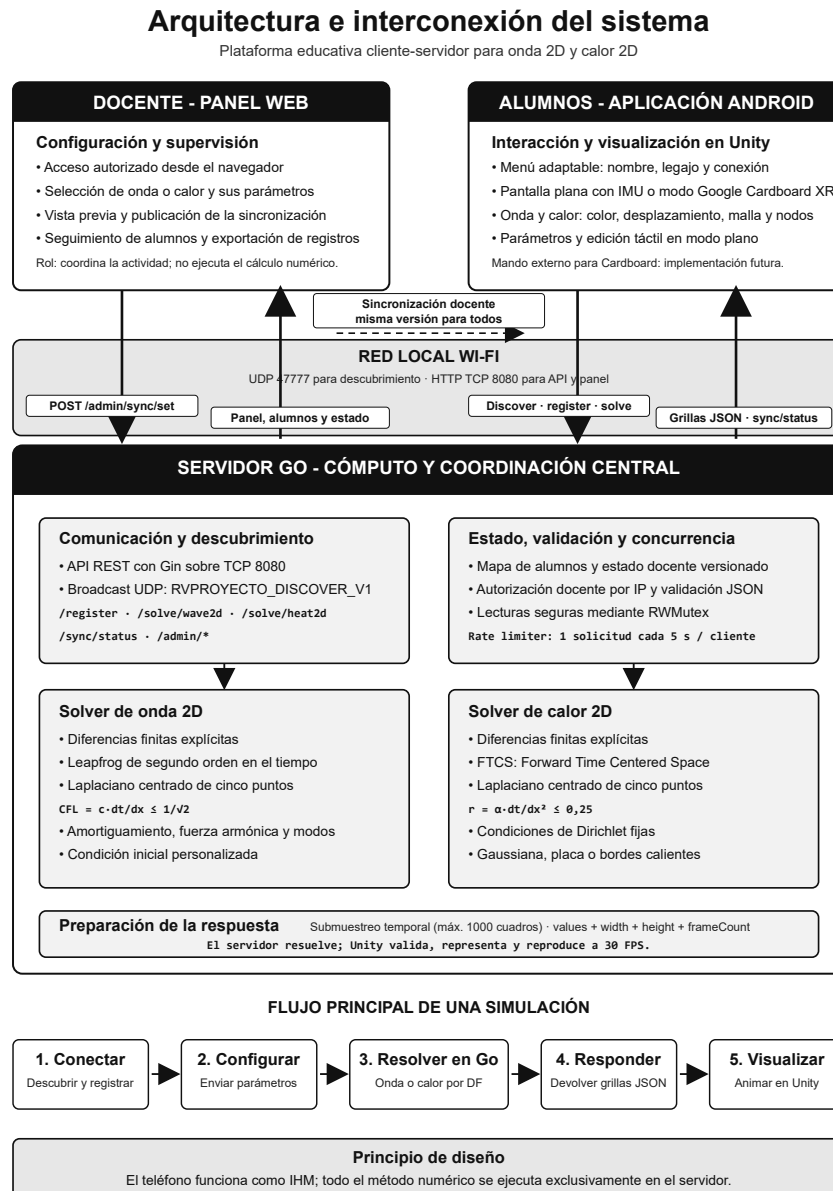


Figura 1: Arquitectura e interconexión de la plataforma. El servidor centraliza la comunicación, la sincronización y la resolución numérica; Unity representa las grillas recibidas y utiliza el teléfono como dispositivo IHM.

5. Arquitectura del Servidor

El servidor está escrito íntegramente en Go y utiliza el marco de desarrollo (*framework*) **Gin** para la gestión de rutas HTTP [2, 3]. El código se organiza en nueve archivos, cada uno con una responsabilidad bien delimitada:

Archivo	Responsabilidad
<code>main.go</code>	Punto de entrada: flags CLI, GUI de lanzador y router
<code>gui.go</code>	Lanzador gráfico en navegador; ciclo de vida del servidor
<code>auth.go</code>	Autenticación del profesor y limitación de tasa por IP
<code>sync.go</code>	Estado global en memoria: clientes, sincronización y versión
<code>handlers.go</code>	Endpoints <code>/solve</code> , <code>/register</code> y <code>/sync/status</code>
<code>admin.go</code>	Endpoints del panel web docente (<code>/admin/*</code>)
<code>wave_api.go</code>	Endpoint <code>/solve/wave2d</code> dedicado a la app Unity (onda)
<code>heat_api.go</code>	Endpoint <code>/solve/heat2d</code> dedicado a la app Unity (calor)
<code>solver.go</code>	Solvers numéricos de EDPs (sin dependencias HTTP)

Todos los archivos pertenecen al mismo paquete `main`, lo que significa que comparten tipos, variables y funciones sin necesidad de importaciones entre ellos. El estado global del servidor vive en una única variable (`estado`) definida en `sync.go`, y todos los handlers acceden a ella mediante sus métodos.

5.1. Resumen de rutas

La tabla siguiente sintetiza todos los endpoints expuestos por el servidor, el método HTTP, quién puede usarlos y qué devuelven.

Ruta	Método	Acceso	Función / Respuesta
/	GET	Todos	Sirve el login del profesor (<code>login.html</code>).
/auth	POST	Todos	Verifica el código y autoriza su IP. Devuelve <code>{ok:true}</code> .
/panel	GET	Profesor	Sirve el panel docente; redirige al login si la IP no está autorizada.
/register	POST	App móvil	Registra o actualiza un alumno. Devuelve <code>{ok:true}</code> .
/solve	GET	Panel web	Calcula la EDP y devuelve <code>{sync, ecuacion, result}</code> con hasta 1000 frames.
/solve/wave2d	POST	App Unity	Acepta parámetros de onda, amortiguamiento, excitación y condición personalizable; devuelve frames aplanados.
/solve/heat2d	POST	App Unity	Acepta parámetros térmicos y devuelve frames en el mismo formato aplanado.
/sync/status	GET	App móvil	Devuelve estado y versión de sincronización sin ejecutar el solver.
/admin/clients	GET	Profesor	Lista alumnos con IP, legajo, nombre y último request.
/admin/sync/set	POST	Profesor	Activa la sincronización, incluido amortiguamiento y excitación.
/admin/sync/clear	POST	Profesor	Desactiva la sincronización.

5.2. main.go — Punto de entrada y router

`main.go` declara la constante del código de acceso del profesor, interpreta el flag de línea de comandos y decide entre arrancar la GUI de lanzador o el servidor directamente. Las rutas se registran en `setupRouter()`, separando la configuración del router de su ciclo de vida.

Sin el flag `-port`, `main` llama a `runLauncherGUI()` (definida en `gui.go`), que levanta un servidor HTTP local en un puerto efímero, abre el navegador en esa dirección y queda a la espera: el usuario elige el puerto de red y presiona “Lanzar”. Con el flag, el servidor arranca directamente sin GUI, lo que facilita el uso en scripts o despliegues automatizados.

`setupRouter()` centraliza el registro de todas las rutas. Existen tres endpoints de cálculo: `/solve` para el panel web (array 3D), y `/solve/wave2d` y `/solve/heat2d` para Unity (arrays 1D aplanados, compatibles con `JsonUtility`). Los tres pasan por el rate limiter. El nuevo endpoint `/sync/status` permite a la app consultar el estado de sincronización sin ejecutar el solver, reduciendo el costo de las consultas de *polling*.

5.3. auth.go — Autenticación y limitación de tasa

Este archivo implementa dos mecanismos de control de acceso independientes: la autenticación del profesor por IP y el limitador de tasa para los endpoints de cálculo.

`ipsAutorizadas` actúa como lista blanca de IPs del profesor, protegido por su propio `sync.RWMutex`. `limiters` es un `sync.Map` — mapa concurrente sin mutex manual — que almacena un limitador de tasa por IP.

```
1 package main
2
3 import (
4     "flag"
5     "github.com/gin-gonic/gin"
6 )
7
8 const codigoProfesor = "123"
9
10 func main() {
11     port := flag.String("port", "", "puerto del servidor (omitir para
        usar la GUI)")
12     flag.Parse()
13
14     if *port != "" {
15         startGinServer(*port)
16     } else {
17         runLauncherGUI()
18     }
19 }
20
21 func setupRouter() *gin.Engine {
22     router := gin.Default()
23
24     router.StaticFile("/", "./web/login.html")
25     router.StaticFile("/favicon.ico", "./web/favicon.ico")
26     // ... otros archivos estaticos de la webapp ...
27     router.POST("/auth", authHandler)
28     router.GET("/panel", panelHandler)
29
30     admin := router.Group("/admin")
31     {
32         admin.GET("/clients", clientesHandler)
33         admin.POST("/sync/clear", syncClearHandler)
34         admin.POST("/sync/set", syncSetAdminHandler)
35     }
36
37     router.POST("/register", registerHandler)
38     router.GET("/sync/status", syncStatusHandler)
39     router.POST("/solve/wave2d", RateLimitMiddleware(),
        waveSolveUnityHandler)
40     router.POST("/solve/heat2d", RateLimitMiddleware(),
        heatSolveUnityHandler)
41     router.GET("/solve", RateLimitMiddleware(), solveHandler)
42
43     return router
44 }
```

Código 1: main.go completo

```
1 package main
2
3 import (
4     "fmt"
5     "math"
6     "net/http"
7     "sync"
8     "time"
9
10    "github.com/gin-gonic/gin"
11    "golang.org/x/time/rate"
12 )
13
14 type authRequest struct {
15     Codigo string `json:"codigo"`
16 }
17
18 var (
19     ipsAutorizadas = make(map[string]bool)
20     ipsAutorizadasMu sync.RWMutex
21 )
22
23 var limiters sync.Map
```

Código 2: auth.go — imports y variables globales

El middleware lee primero la cabecera `X-Forwarded-For` antes de caer en `c.Client-IP()`: esto permite identificar la IP real del cliente cuando el servidor corre detrás de un proxy o balanceador de carga. El limitador usa el patrón *token bucket*: un token se regenera cada cinco segundos. Si el token no está disponible, se calcula el tiempo de espera, se cancela la reserva (para no consumirlo) y se responde 429 con la cabecera `Retry-After`.

`panelHandler` redirige al login en lugar de devolver un error HTTP, para que el flujo sea transparente en el navegador del profesor.

5.4. sync.go — Estado global en memoria

Este archivo centraliza todo el estado compartido entre handlers. El sistema de roles de asistente fue eliminado: `Cliente` ya no tiene campo `Rol`, y `parametrosSync` se extendió para incluir los parámetros de amortiguamiento y excitación forzada.

Los campos de `parametrosSync` permiten que el profesor sincronice no solo la ecuación y sus parámetros básicos, sino también el amortiguamiento viscoso (γ) y la excitación forzada (frecuencia y forma). El campo `version` es un contador monótonamente creciente: se incrementa en cada `ActivarSync` y `DesactivarSync`, lo que permite a los clientes detectar un cambio con una consulta liviana a `/sync/status` sin tener que re-ejecutar el solver.

`RegistrarCliente` es idempotente: si la IP ya existe, actualiza legajo y nombre preservando el resto del estado. Las operaciones de lectura usan `RLock` (múltiples lectores simultáneos); las de escritura usan `Lock` (exclusividad). `defer` garantiza la liberación del mutex incluso ante un panic.

La sincronización funciona como difusión pasiva: el profesor escribe los parámetros una

```
1 func getLimiter(ip string) *rate.Limiter {
2     if v, ok := limiters.Load(ip); ok {
3         return v.(*rate.Limiter)
4     }
5     lim := rate.NewLimiter(rate.Every(5*time.Second), 1)
6     actual, _ := limiters.LoadOrStore(ip, lim)
7     return actual.(*rate.Limiter)
8 }
9
10 func RateLimitMiddleware() gin.HandlerFunc {
11     return func(c *gin.Context) {
12         ip := c.GetHeader("X-Forwarded-For")
13         if ip == "" {
14             ip = c.ClientIP()
15         }
16         lim := getLimiter(ip)
17
18         res := lim.Reserve()
19         if !res.OK() {
20             c.AbortWithStatusJSON(http.StatusTooManyRequests,
21                 gin.H{"error": "Demasiadas peticiones"})
22             return
23         }
24
25         delay := res.Delay()
26         if delay <= 0 {
27             c.Next()
28             return
29         }
30
31         res.Cancel()
32         sec := int(math.Ceil(delay.Seconds()))
33         c.Header("Retry-After", fmt.Sprintf("%d", sec))
34         c.AbortWithStatusJSON(http.StatusTooManyRequests, gin.H{
35             "error": "Demasiadas peticiones",
36             "waitSeconds": sec,
37             "message": fmt.Sprintf("Espera %ds antes de reintentar",
38                 sec),
39         })
40     }
41 }
```

Código 3: Limitador de tasa por IP


```
1 func autorizarIP(ip string) {
2     ipsAutorizadasMu.Lock()
3     defer ipsAutorizadasMu.Unlock()
4     ipsAutorizadas[ip] = true
5     fmt.Println("[auth] IP autorizada:", ip)
6 }
7
8 func ipAutorizada(ip string) bool {
9     ipsAutorizadasMu.RLock()
10    defer ipsAutorizadasMu.RUnlock()
11    return ipsAutorizadas[ip]
12 }
13
14 func authHandler(c *gin.Context) {
15     var req authRequest
16     if err := c.ShouldBindJSON(&req); err != nil {
17         c.JSON(http.StatusBadRequest,
18             gin.H{"error": "Cuerpo de solicitud invalido"})
19         return
20     }
21     if req.Codigo != codigoProfesor {
22         c.JSON(http.StatusUnauthorized, gin.H{"error": "Codigo
23             incorrecto"})
24         return
25     }
26     autorizarIP(c.ClientIP())
27     c.JSON(http.StatusOK, gin.H{"ok": true})
28 }
29
30 func panelHandler(c *gin.Context) {
31     if !ipAutorizada(c.ClientIP()) {
32         c.Redirect(http.StatusFound, "/")
33         return
34     }
35     c.File("./web/panel.html")
36 }
```

Código 4: Autenticación del profesor

```
1 package main
2
3 import (
4     "sync"
5     "time"
6 )
7
8 type Cliente struct {
9     IP          string
10    Legajo       string
11    Nombre       string
12    UltimoRequest time.Time
13 }
14
15 type parametrosSync struct {
16     Ecuacion      string
17     L             float64
18     T             float64
19     C             float64
20     Inicial       string
21     Amortiguamiento bool
22     Gamma         float64
23     Excitacion    bool
24     FormaExcitacion string
25     FrecExcitacion float64
26 }
27
28 type estadoGlobal struct {
29     mu          sync.RWMutex
30     clientes    map[string]*Cliente
31     sync        struct {
32         activo      bool
33         parametros parametrosSync
34         version     uint64 // incrementa en cada ActivarSync /
35                        DesactivarSync
36     }
37 }
38
39 var estado = &estadoGlobal{
40     clientes: make(map[string]*Cliente),
41 }
```

Código 5: sync.go — tipos y estado global

```
1 func (e *estadoGlobal) RegistrarCliente(ip, legajo, nombre string) {
2     e.mu.Lock()
3     defer e.mu.Unlock()
4
5     if c, existe := e.clientes[ip]; existe {
6         c.Legajo = legajo
7         c.Nombre = nombre
8         c.UltimoRequest = time.Now()
9         return
10    }
11
12    e.clientes[ip] = &Cliente{
13        IP:            ip,
14        Legajo:        legajo,
15        Nombre:        nombre,
16        UltimoRequest: time.Now(),
17    }
18 }
19
20 func (e *estadoGlobal) ActualizarRequest(ip string) {
21     e.mu.Lock()
22     defer e.mu.Unlock()
23     if c, existe := e.clientes[ip]; existe {
24         c.UltimoRequest = time.Now()
25     }
26 }
27
28 func (e *estadoGlobal) Clientes() []*Cliente {
29     e.mu.RLock()
30     defer e.mu.RUnlock()
31     lista := make([]*Cliente, 0, len(e.clientes))
32     for _, c := range e.clientes {
33         lista = append(lista, c)
34     }
35     return lista
36 }
```

Código 6: Registro y consulta de clientes

```
1 func (e *estadoGlobal) ActivarSync(p parametrosSync) {
2     e.mu.Lock()
3     defer e.mu.Unlock()
4     e.sync.activo = true
5     e.sync.parametros = p
6     e.sync.version++
7 }
8
9 func (e *estadoGlobal) DesactivarSync() {
10    e.mu.Lock()
11    defer e.mu.Unlock()
12    e.sync.activo = false
13    e.sync.parametros = parametrosSync{}
14    e.sync.version++
15 }
16
17 func (e *estadoGlobal) EstadoSync() (bool, parametrosSync) {
18    e.mu.RLock()
19    defer e.mu.RUnlock()
20    return e.sync.activo, e.sync.parametros
21 }
22
23 // EstadoSyncVersionado permite detectar cambios sin re-ejecutar el
24 // solver.
25 func (e *estadoGlobal) EstadoSyncVersionado() (bool, parametrosSync,
26    uint64) {
27    e.mu.RLock()
28    defer e.mu.RUnlock()
29    return e.sync.activo, e.sync.parametros, e.sync.version
30 }
```

Código 7: Control de sincronizacion

vez, y cualquier cliente que llame a `/solve`, `/solve/wave2d` o `/solve/heat2d` recibe esos parámetros en lugar de los propios. No hay websockets ni notificaciones push. El campo `"sync": true` en la respuesta indica al cliente que los parámetros fueron impuestos. La app puede además consultar `GET /sync/status` para obtener el campo `"version"`: si el número cambió respecto al último guardado, hay una nueva configuración que descargar.

5.5. handlers.go — Endpoints del panel web

Este archivo implementa el registro de alumnos, el endpoint `/solve` utilizado por el panel web y el nuevo `/sync/status`. El solver de onda 2D acepta parámetros adicionales de amortiguamiento y excitación forzada, y el panel puede seleccionar modos de vibración superiores mediante los query params `modoA` y `modoB` (codificados internamente como `"modo:m1:m2"`).

```
1 package main
2
3 import (
4     "net/http"
5     "strconv"
6     "github.com/gin-gonic/gin"
7 )
8
9 type registerRequest struct {
10     Legajo string `json:"legajo"`
11     Nombre string `json:"nombre"`
12 }
13
14 func registerHandler(c *gin.Context) {
15     var req registerRequest
16     if err := c.ShouldBindJSON(&req); err != nil {
17         c.JSON(http.StatusBadRequest,
18             gin.H{"error": "Cuerpo de solicitud invalido"})
19         return
20     }
21     if req.Legajo == "" || req.Nombre == "" {
22         c.JSON(http.StatusBadRequest,
23             gin.H{"error": "Legajo y nombre son obligatorios"})
24         return
25     }
26     estado.RegistrarCliente(c.ClientIP(), req.Legajo, req.Nombre)
27     c.JSON(http.StatusOK, gin.H{"ok": true})
28 }
```

Código 8: handlers.go — registro de alumno

Los parámetros opcionales `gamma` (amortiguamiento) y `frecExcitacion` (excitación forzada) se leen solo si están presentes en la URL: si el valor de `gamma` es mayor que cero se activa el amortiguamiento; si `frecExcitacion` es válido se construye el struct `excitacionOpts` con la frecuencia y la forma de onda indicadas. Cuando `inicial=modo`, los query params `modoA` y `modoB` especifican los índices del modo; ambos se validan en el rango `[1, 5]` antes de codificarse como `"modo:m1:m2"`.

```
1 func solveHandler(c *gin.Context) {
2     estado.ActualizarRequest(c.ClientIP())
3     syncActivo, params := estado.EstadoSync()
4
5     var L, T, cw, gamma float64
6     var inicial, ecuacion string
7     var amortiguamiento bool
8     var exc excitacionOpts
9
10    if syncActivo {
11        L = params.L
12        T = params.T
13        cw = params.C
14        inicial = params.Inicial
15        ecuacion = params.Ecuacion
16        amortiguamiento = params.Amortiguamiento
17        gamma = params.Gamma
18        exc = excitacionOpts{
19            Activa: params.Excitacion,
20            Forma: params.FormaExcitacion,
21            Frecuencia: params.FrecExcitacion,
22        }
23    }
24    // Si no hay sincronizacion, se leen los datos del alumno.
25 }
```

Código 9: Inicio del cálculo y aplicación de la sincronización

`syncStatusHandler` (en el mismo archivo, no listado aquí por brevedad) atiende `GET /sync/status`: llama a `EstadoSyncVersionado()` y devuelve el estado completo junto con el número de versión, sin ejecutar ningún solver.

5.6. `wave_api.go` — Endpoint dedicado a Unity

Este archivo implementa el endpoint `POST /solve/wave2d`, diseñado específicamente para la aplicación Unity. La diferencia principal respecto a `/solve` es el formato de respuesta: Unity usa `JsonUtility`, que no puede deserializar arrays de arrays anidados, por lo que los frames se aplanan en un único array 1D.

`waveGridResponse` aplanar los frames en un array `Values` de tipo `[]float32` (precisión simple para reducir el tamaño de la respuesta JSON). El valor en la posición $(frame, x, y)$ se obtiene con el índice

`frame*width*height + x*height + y`. `waveParametersResponse` devuelve los parámetros efectivamente usados, que pueden diferir de los solicitados si hay sincronización activa.

La sincronización sobrescribe todos los parámetros del request, incluyendo los campos de condición inicial personalizada (`InitialWidth`, `InitialHeight`, `InitialValues`), que se anulan porque la sincronización no admite grillas dibujadas por el usuario.

La condición inicial "personalizada" permite a Unity enviar una grilla dibujada por el usuario (hasta 128x128). El servidor valida que las dimensiones y valores sean correctos y que no haya NaN ni Inf antes de pasarlos al solver, ya que estos valores romperían el esquema leapfrog.

```
1 if !syncActivo {
2     // Continuación de solveHandler.
3     Ls, ok1 := c.GetQuery("L")
4     Ts, ok2 := c.GetQuery("T")
5     cs, ok3 := c.GetQuery("c")
6     inicial, _ = c.GetQuery("inicial")
7     ecuacion, _ = c.GetQuery("ecuacion")
8
9     if !ok1 || !ok2 || !ok3 {
10         c.JSON(http.StatusBadRequest,
11             gin.H{"error": "Faltan parametros en la consulta"})
12         return
13     }
14
15     var err1, err2, err3 error
16     L, err1 = strconv.ParseFloat(Ls, 64)
17     T, err2 = strconv.ParseFloat(Ts, 64)
18     cw, err3 = strconv.ParseFloat(cs, 64)
19
20     if err1 != nil || err2 != nil || err3 != nil {
21         c.JSON(http.StatusBadRequest, gin.H{"error": "Parametros
22             invalidos"})
23         return
24     }
25
26     lMax := 10.0
27     if ecuacion == "calor2d" { lMax = 20.0 }
28     if L < 1 || L > lMax || T < 1 || T > 100 || cw < 0.1 || cw > 2 {
29         c.JSON(http.StatusBadRequest,
30             gin.H{"error": "Parametros fuera de rango"})
31         return
32     }
33
34     if gs, ok := c.GetQuery("gamma"); ok {
35         if g, err := strconv.ParseFloat(gs, 64); err == nil &&
36             g >= 0 && g <= 1 {
37             gamma = g
38             amortiguamiento = gamma > 0
39         }
40     }
41
42     if fs, ok := c.GetQuery("frecExcitacion"); ok {
43         if f, err := strconv.ParseFloat(fs, 64); err == nil &&
44             f > 0 && f <= 20 {
45             forma, _ := c.GetQuery("formaExcitacion")
46             if forma == "" { forma = "seno" }
47             exc = excitacionOpts{Activa: true, Forma: forma,
48                 Frecuencia: f}
49         }
50     }
51 }
```

Código 10: Lectura y validación de los parámetros de cálculo

```
1 // Tramo final de solveHandler.
2 var result interface{}
3 switch ecuacion {
4 case "calor2d":
5     result = ecuacion_calor_2d(L, T, cw, inicial)
6 default:
7     ecuacion = "onda2d"
8     effectiveGamma := 0.0
9     if amortiguamiento { effectiveGamma = gamma }
10    result = ecuacion_onda_2d(L, T, cw, effectiveGamma, exc, inicial
11    )
12
13    c.JSON(http.StatusOK, gin.H{
14        "sync":      syncActivo,
15        "ecuacion":  ecuacion,
16        "result":    result,
17    })
```

Código 11: Selección del solver y construcción de la respuesta

`flattenWaveFrames` convierte el array 3D `[frame][x][y]` en un vector 1D en orden row-major. Unity reconstruye las coordenadas con `values[frame*width*height + x*height + y]`.

5.7. `heat_api.go` — Endpoint de calor dedicado a Unity

Este archivo implementa `POST /solve/heat2d`, el equivalente de `/solve/wave2d` pero para la ecuación de calor. La estructura de la respuesta es idéntica a la de onda (usa el mismo `waveGridResponse` y `flattenWaveFrames`), lo que simplifica el cliente Unity: ambas ecuaciones se deserializan con el mismo código.

A diferencia del endpoint de onda, la ecuación de calor no admite condición inicial personalizada (no existe dibujo interactivo para el calor en la app Unity) ni excitación forzada. La sincronización solo se activa si la ecuación sincronizada es `calor2d`: un alumno que solicita calor no recibe los parámetros de onda del profesor aunque la sincronización esté activa.

5.8. `admin.go` — Endpoints del panel docente

Este archivo implementa las rutas del grupo `/admin`. El sistema de roles fue eliminado: ya no existen `promoverHandler` ni `demoverHandler`. El struct `syncSetRequest` se movió aquí desde `handlers.go` y se extendió con los campos de amortiguamiento y excitación.

`clienteResponse` ya no expone el campo `Rol` porque el sistema de roles fue eliminado. La separación entre el struct interno `Cliente` y el de respuesta sigue siendo útil: `time.Time` se formatea como `HH:MM:SS` antes de serializar, sin exponer el tipo interno a la estructura pública de la API.

`syncSetAdminHandler` propaga todos los parámetros de la simulación incluyendo amortiguamiento y excitación. El límite de `T` subió a 100 en línea con el cambio en


```
1 package main
2
3 import (
4     "math"
5     "net/http"
6     "github.com/gin-gonic/gin"
7 )
8
9 type waveSolveRequest struct {
10     L          float64    `json:"l"`
11     T          float64    `json:"t"`
12     C          float64    `json:"c"`
13     Inicial    string     `json:"inicial"`
14     Amortiguamiento bool      `json:"amortiguamiento"`
15     Gamma      float64    `json:"gamma"`
16     Excitacion  bool      `json:"excitacion"`
17     FormaExcitacion string    `json:"formaExcitacion"`
18     FrecExcitacion float64   `json:"frecExcitacion"`
19     InitialWidth int       `json:"initialWidth"`
20     InitialHeight int      `json:"initialHeight"`
21     InitialValues []float64 `json:"initialValues"`
22 }
23
24 type waveParametersResponse struct {
25     L          float64    `json:"l"`
26     T          float64    `json:"t"`
27     C          float64    `json:"c"`
28     Inicial    string     `json:"inicial"`
29     Amortiguamiento bool      `json:"amortiguamiento"`
30     Gamma      float64    `json:"gamma"`
31 }
32
33 type waveGridResponse struct {
34     Width      int       `json:"width"`
35     Height     int       `json:"height"`
36     FrameCount int      `json:"frameCount"`
37     Values     []float32 `json:"values"`
38 }
39
40 type waveSolveResponse struct {
41     Sync      bool      `json:"sync"`
42     Ecuacion  string    `json:"ecuacion"`
43     Parametros waveParametersResponse `json:"parametros"`
44     Result    waveGridResponse `json:"result"`
45 }
```

Código 12: wave_api.go — estructuras de entrada y salida

```
1 func waveSolveUnityHandler(c *gin.Context) {
2     estado.ActualizarRequest(c.ClientIP())
3
4     var req waveSolveRequest
5     if err := c.ShouldBindJSON(&req); err != nil {
6         c.JSON(http.StatusBadRequest, gin.H{"error": "Cuerpo de
7             solicitud invalido"})
8         return
9     }
10    syncActivo, syncParams := estado.EstadoSync()
11    if syncActivo {
12        req.L           = syncParams.L
13        req.T           = syncParams.T
14        req.C           = syncParams.C
15        req.Inicial     = syncParams.Inicial
16        req.Amortiguamiento = syncParams.Amortiguamiento
17        req.Gamma       = syncParams.Gamma
18        req.Excitacion  = syncParams.Excitacion
19        req.FormaExcitacion = syncParams.FormaExcitacion
20        req.FrecExcitacion = syncParams.FrecExcitacion
21        req.InitialWidth  = 0
22        req.InitialHeight = 0
23        req.InitialValues = nil
24    }
25
26    if req.L < 1 || req.L > 10 || req.T < 1 || req.T > 100 ||
27        req.C < 0.1 || req.C > 2 {
28        c.JSON(http.StatusBadRequest, gin.H{"error": "Parametros fuera
29            de rango"})
30        return
31    }
```

Código 13: wave_api.go — handler principal

```
1  switch req.Inicial {
2  case "seno", "triangular", "gauss":
3      // valido
4  case "personalizada":
5      if req.InitialWidth < 2 || req.InitialWidth > 128 ||
6          req.InitialHeight < 2 || req.InitialHeight > 128 ||
7          len(req.InitialValues) != req.InitialWidth*req.InitialHeight
8          {
9          c.JSON(http.StatusBadRequest,
10             gin.H{"error": "Grilla inicial personalizada invalida"})
11             return
12         }
13     for _, value := range req.InitialValues {
14         if math.IsNaN(value) || math.IsInf(value, 0) ||
15             value < -2 || value > 2 {
16             c.JSON(http.StatusBadRequest,
17                 gin.H{"error": "Valores iniciales fuera de rango"})
18                 return
19             }
20         }
21     default:
22         c.JSON(http.StatusBadRequest,
23             gin.H{"error": "Condicion inicial invalida"})
24         return
25     }
```

Código 14: Validación de la condición inicial de la onda

`handlers.go`. `syncClearHandler` no requiere validación: desactivar una sincronización inexistente es inocuo.

5.9. `gui.go` — Lanzador gráfico

Para simplificar el despliegue en un entorno de aula sin acceso a una terminal, `gui.go` implementa un servidor HTTP local efímero (en un puerto aleatorio asignado por el sistema operativo) que sirve una página de configuración y gestiona el ciclo de vida del servidor Gin.

Al ejecutar el binario sin flags, `runLauncherGUI()` levanta este servidor auxiliar, abre el navegador en su dirección y bloquea indefinidamente. Desde la página, el usuario puede:

- Ver las IPs locales detectadas (para comunicarlasy a los alumnos).
- Elegir el puerto de red del servidor Gin.
- Lanzar o detener el servidor con un clic.

El servidor Gin corre en una goroutine separada iniciada por `startGinServer()` (también en `gui.go`). El estado de ejecución se protege con un mutex para evitar lanzamientos dobles. La página hace polling cada dos segundos a `/status` para actualizar el indicador visual; cuando el servidor se detiene (ya sea por la GUI o externamente), la interfaz refleja el cambio.

```
1 effectiveGamma := 0.0
2 if req.Amortiguamiento && req.Gamma >= 0 && req.Gamma <= 1 {
3     effectiveGamma = req.Gamma
4 }
5
6 exc := excitacionOpts{}
7 if req.Excitacion && req.FrecExcitacion > 0 && req.FrecExcitacion <=
8     20 {
9     forma := req.FormaExcitacion
10    if forma == "" { forma = "seno" }
11    exc = excitacionOpts{Activa: true, Forma: forma, Frecuencia: req
12        .FrecExcitacion}
13 }
14
15 var frames [][][]float64
16 if req.Inicial == "personalizada" {
17     frames = ecuacion_onda_2d_personalizada(
18         req.L, req.T, req.C, effectiveGamma, exc,
19         req.InitialValues, req.InitialWidth, req.InitialHeight,
20     )
21 } else {
22     frames = ecuacion_onda_2d(req.L, req.T, req.C, effectiveGamma,
23         exc, req.Inicial)
24 }
25 result := flattenWaveFrames(frames)
26
27 c.JSON(http.StatusOK, waveSolveResponse{
28     Sync:      syncActivo,
29     Ecuacion:   "onda2d",
30     Parametros: waveParametersResponse{
31         L: req.L, T: req.T, C: req.C, Inicial: req.Inicial,
32         Amortiguamiento: req.Amortiguamiento, Gamma: effectiveGamma,
33     },
34     Result: result,
35 })
36 }
```

Código 15: Cálculo y construcción de la respuesta de onda

```

1 func flattenWaveFrames(frames [][][]float64) waveGridResponse {
2     if len(frames) == 0 || len(frames[0]) == 0 || len(frames[0][0]) == 0
3     {
4         return waveGridResponse{Values: make([]float32, 0)}
5     }
6
7     width := len(frames[0])
8     height := len(frames[0][0])
9     values := make([]float32, 0, len(frames)*width*height)
10
11     for _, frame := range frames {
12         for x := 0; x < width; x++ {
13             for y := 0; y < height; y++ {
14                 values = append(values, float32(frame[x][y]))
15             }
16         }
17     }
18
19     return waveGridResponse{
20         Width:      width,
21         Height:     height,
22         FrameCount:  len(frames),
23         Values:      values,
24     }
25 }

```

Código 16: Aplanamiento de frames para Unity

```

1 type heatSolveRequest struct {
2     L      float64 `json:"l"`
3     T      float64 `json:"t"`
4     Alpha  float64 `json:"alpha"`
5     Inicial string  `json:"inicial"`
6 }
7
8 type heatSolveResponse struct {
9     Sync      bool           `json:"sync"`
10    Ecuacion   string        `json:"ecuacion"`
11    Parametros heatParametersResponse `json:"parametros"`
12    Result     waveGridResponse `json:"result"`
13 }

```

Código 17: Estructuras de entrada y salida para la ecuación de calor

```
1 func heatSolveUnityHandler(c *gin.Context) {
2     estado.ActualizarRequest(c.ClientIP())
3
4     var req heatSolveRequest
5     if err := c.ShouldBindJSON(&req); err != nil {
6         c.JSON(http.StatusBadRequest, gin.H{"error": "Cuerpo invalido"})
7         return
8     }
9
10    // Si la sincronizacion esta activa y es para calor, sobrecribir
11    // parametros
12    syncActivo, syncParams := estado.EstadoSync()
13    heatSync := syncActivo && syncParams.Ecuacion == "calor2d"
14    if heatSync {
15        req.L = syncParams.L; req.T = syncParams.T
16        req.Alpha = syncParams.C
17        req.Inicial = syncParams.Inicial
18    }
19
20    if req.L < 1 || req.L > 20 || req.T < 1 || req.T > 100 ||
21    req.Alpha < 0.1 || req.Alpha > 2 {
22        c.JSON(http.StatusBadRequest, gin.H{"error": "Parametros fuera
23        de rango"})
24        return
25    }
26
27    switch req.Inicial {
28    case "gauss", "bordes", "placa":
29    default:
30        c.JSON(http.StatusBadRequest,
31            gin.H{"error": "Condicion inicial invalida"})
32        return
33    }
34
35    frames := ecuacion_calor_2d(req.L, req.T, req.Alpha, req.Inicial)
36    c.JSON(http.StatusOK, heatSolveResponse{
37        Sync: heatSync, Ecuacion: "calor2d",
38        Parametros: heatParametersResponse{
39            L: req.L, T: req.T, Alpha: req.Alpha, Inicial: req.Inicial,
40        },
41        Result: flattenWaveFrames(frames),
42    })
43 }
```

Código 18: Validación, cálculo y respuesta de la ecuación de calor

```
1 package main
2
3 import (
4     "net/http"
5     "github.com/gin-gonic/gin"
6 )
7
8 type clienteResponse struct {
9     IP            string `json:"ip"`
10    Legajo         string `json:"legajo"`
11    Nombre        string `json:"nombre"`
12    UltimoRequest string `json:"ultimoRequest"`
13 }
14
15 func clientesHandler(c *gin.Context) {
16     clientes := estado.Clientes()
17     lista := make([]clienteResponse, 0, len(clientes))
18     for _, cl := range clientes {
19         lista = append(lista, clienteResponse{
20             IP:            cl.IP,
21             Legajo:        cl.Legajo,
22             Nombre:        cl.Nombre,
23             UltimoRequest: cl.UltimoRequest.Format("15:04:05"),
24         })
25     }
26     c.JSON(http.StatusOK, gin.H{"clientes": lista})
27 }
```

Código 19: admin.go — lista de clientes

```

1 type syncSetRequest struct {
2     Ecuacion      string  `json:"ecuacion"`
3     L              float64  `json:"L"`
4     T              float64  `json:"T"`
5     C              float64  `json:"c"`
6     Inicial        string  `json:"inicial"`
7     Amortiguamiento bool    `json:"amortiguamiento"`
8     Gamma          float64  `json:"gamma"`
9     Excitacion      bool    `json:"excitacion"`
10    FormaExcitacion string  `json:"formaExcitacion"`
11    FrecExcitacion   float64  `json:"frecExcitacion"`
12 }
13
14 func syncClearHandler(c *gin.Context) {
15     estado.DesactivarSync()
16     c.JSON(http.StatusOK, gin.H{"ok": true})
17 }
18
19 func syncSetAdminHandler(c *gin.Context) {
20     var req syncSetRequest
21     if err := c.ShouldBindJSON(&req); err != nil {
22         c.JSON(http.StatusBadRequest,
23             gin.H{"error": "Cuerpo de solicitud invalido"})
24         return
25     }
26     lMax := 10.0
27     if req.Ecuacion == "calor2d" { lMax = 20.0 }
28     if req.L < 1 || req.L > lMax || req.T < 1 || req.T > 100 ||
29         req.C < 0.1 || req.C > 2 {
30         c.JSON(http.StatusBadRequest, gin.H{"error": "Parametros fuera
31             de rango"})
32         return
33     }
34     estado.ActivarSync(parametrosSync{
35         Ecuacion:      req.Ecuacion,
36         L:              req.L,
37         T:              req.T,
38         C:              req.C,
39         Inicial:        req.Inicial,
40         Amortiguamiento: req.Amortiguamiento,
41         Gamma:          req.Gamma,
42         Excitacion:      req.Excitacion,
43         FormaExcitacion: req.FormaExcitacion,
44         FrecExcitacion:  req.FrecExcitacion,
45     })
46     c.JSON(http.StatusOK, gin.H{"ok": true})
47 }

```

Código 20: admin.go — sincronizacion

5.10. solver.go — Implementación de los solvers

Ambos solvers residen en `solver.go`, sin ninguna dependencia del paquete HTTP. A continuación se presenta el código de cada uno.

5.10.1. Solver de la ecuación de onda 2D

```
1 // excitacionOpts agrupa los parametros de la fuente forzada.
2 type excitacionOpts struct {
3     Activa      bool
4     Forma       string // "seno"
5     Frecuencia  float64 // Hz
6 }
7
8 // ecuacion_onda_2d: wrapper publico; delega en
9 // ecuacion_onda_2d_con_inicial.
10 func ecuacion_onda_2d(L, T, c, gamma float64,
11     exc excitacionOpts, inicial string) [][][]float64 {
12     return ecuacion_onda_2d_con_inicial(L, T, c, gamma, exc, inicial,
13         nil, 0, 0)
14 }
15
16 func ecuacion_onda_2d_con_inicial(
17     L, T, c, gamma float64, exc excitacionOpts,
18     inicial string,
19     customValues []float64, customWidth, customHeight int,
20 ) [][][]float64 {
21     n := int(math.Ceil(L * 8))
22     if n < 16 { n = 16 }
23     d := L / float64(n)
24
25     CFL := 0.9 / math.Sqrt2
26     d_t := CFL * d / c
27     n_t := int(math.Ceil(T / d_t))
28     d_t = T / float64(n_t)
29     r := (c * d_t / d) * (c * d_t / d) // r = (c*dt/dx)^2
30
31     maxFrames := 1000
32     step := 1
33     if n_t > maxFrames { step = n_t / maxFrames }
34
35     out := make([][][]float64, 0, maxFrames)
36     pp := newGrid(n) // u^(k-2)
37     p := newGrid(n) // u^(k-1)
38     cur := newGrid(n) // u^k
39
40     modeA, modeB, isMode := parseWaveMode(inicial)
```

Código 21: Parámetros y discretización del solver de onda

5.10.2. Solver de la ecuación de calor 2D

```
1 // Condicion inicial
2 for i := 1; i < n-1; i++ {
3     for j := 1; j < n-1; j++ {
4         x := float64(i) * d; y := float64(j) * d
5         switch {
6             case inicial == "personalizada":
7                 u := float64(i) / float64(n-1)
8                 v := float64(j) / float64(n-1)
9                 pp[i][j] = sampleWaveInitialGrid(customValues,
10                    customWidth, customHeight, u, v)
11             case inicial == "gauss":
12                 pp[i][j] = math.Exp(-math.Pow((x-L/2)/(L/5), 2) -
13                    math.Pow((y-L/2)/(L/5), 2))
14             case inicial == "triangular":
15                 tx := 1 - math.Abs(2*x/L-1)
16                 ty := 1 - math.Abs(2*y/L-1)
17                 pp[i][j] = tx * ty
18             case isMode:
19                 pp[i][j] = math.Sin(float64(modeA)*math.Pi*x/L) *
20                    math.Sin(float64(modeB)*math.Pi*y/L)
21             default: // seno (modo 1,1)
22                 pp[i][j] = math.Sin(math.Pi*x/L) * math.Sin(math.Pi*y/L)
23         }
24     }
25 }
26 out = append(out, copyGrid(pp))
```

Código 22: Construcción de la condición inicial de la onda

```

1 // Factores de amortiguamiento (Ec. \ref{eq:leapfrog-amort})
2 dampM := 1 - gamma*d_t // (1 - gamma*dt)
3 dampD := 1 + gamma*d_t // (1 + gamma*dt)
4
5 // Forma espacial de la excitacion (precalculada, independiente del
6 // tiempo)
7 excShape := newGrid(n)
8 excAmp := 0.15 * d_t * d_t
9 if exc.Activa {
10     for i := 1; i < n-1; i++ {
11         for j := 1; j < n-1; j++ {
12             x := float64(i) * d; y := float64(j) * d
13             excShape[i][j] = math.Sin(float64(modeA)*math.Pi*x/L) *
14                 math.Sin(float64(modeB)*math.Pi*y/L)
15         }
16     }
17 }
18 // Primer paso temporal (velocidad inicial = 0)
19 for i := 1; i < n-1; i++ {
20     for j := 1; j < n-1; j++ {
21         p[i][j] = pp[i][j] + 0.5*r*(pp[i+1][j]+pp[i-1][j]+
22             pp[i][j+1]+pp[i][j-1]-4*pp[i][j])
23     }
24 }
25 if step == 1 { out = append(out, copyGrid(p)) }

```

Código 23: Amortiguamiento, excitación y primer paso temporal

```

1 // Pasos restantes - leapfrog generalizado
2 for t := 2; t < n_t; t++ {
3     tCurrent := float64(t) * d_t
4     forzado := 0.0
5     if exc.Activa {
6         forzado = math.Sin(2 * math.Pi * exc.Frecuencia * tCurrent)
7     }
8     for i := 1; i < n-1; i++ {
9         for j := 1; j < n-1; j++ {
10             cur[i][j] = (2*p[i][j] - dampM*pp[i][j] +
11                 r*(p[i+1][j]+p[i-1][j]+p[i][j+1]+p[i][j-1]-4*p[i][j]
12                 )) +
13                 excAmp*forzado*excShape[i][j]) / dampD
14         }
15     }
16     if t%step == 0 { out = append(out, copyGrid(cur)) }
17     pp, p, cur = p, cur, pp
18 }
19 return out

```

Código 24: Avance temporal del esquema de onda

```

1 func ecuacion_calor_2d(L, T, alpha float64, inicial string) [][][]
   float64 {
2     n := int(math.Ceil(L * 5))
3     if n < 10 { n = 10 }
4     d := L / float64(n)
5
6     r_max := 0.20
7     d_t := r_max * d * d / alpha
8     n_t := int(math.Ceil(T / d_t))
9     d_t = T / float64(n_t)
10    r := alpha * d_t / (d * d)
11
12    maxFrames := 1000
13    step := 1
14    if n_t > maxFrames { step = n_t / maxFrames }
15
16    out := make([][][]float64, 0, maxFrames)
17    prev := newGrid(n)
18    curr := newGrid(n)
19
20    bcVal := 0.0
21    if inicial == "bordes" { bcVal = 1.0 }

```

Código 25: Parámetros y discretización del solver de calor

```

1 // Condicion inicial
2 for i := 1; i < n-1; i++ {
3     for j := 1; j < n-1; j++ {
4         x := float64(i) * d; y := float64(j) * d
5         switch inicial {
6             case "gauss":
7                 prev[i][j] = math.Exp(-(math.Pow((x-L/2)/(L/5), 2) +
8                                     math.Pow((y-L/2)/(L/5), 2)))
9             case "placa":
10                prev[i][j] = 1.0 // placa caliente: interior caliente,
11                               // bordes frios
12             case "bordes":
13                prev[i][j] = 0.0 // interior frio; calor entra desde los
14                               // extremos
15             default:
16                prev[i][j] = math.Sin(math.Pi*x/L) * math.Sin(math.Pi*y/
17                               L)
18            }
19        }
20    }
21    // Aplicar condicion de contorno inicial
22    for k := 0; k < n; k++ {
23        prev[k][0] = bcVal; prev[k][n-1] = bcVal
24        prev[0][k] = bcVal; prev[n-1][k] = bcVal
25    }
26    out = append(out, copyGrid(prev))

```

Código 26: Condición inicial y contorno de la placa térmica

```
1 // Avance temporal FTCS
2 for t := 1; t < n_t; t++ {
3     for i := 1; i < n-1; i++ {
4         for j := 1; j < n-1; j++ {
5             curr[i][j] = prev[i][j] + r*(prev[i+1][j]+prev[i-1][j]+
6                                     prev[i][j+1]+prev[i][j-1]-4*prev[i][j])
7         }
8     }
9     // Reimposicion de condicion de contorno
10    for k := 0; k < n; k++ {
11        curr[k][0] = bcVal; curr[k][n-1] = bcVal
12        curr[0][k] = bcVal; curr[n-1][k] = bcVal
13    }
14    if t%step == 0 { out = append(out, copyGrid(curr)) }
15    prev, curr = curr, prev
16 }
17 return out
18 }
```

Código 27: Avance temporal FTCS de la ecuación de calor

5.11. Consideraciones de eficiencia

Ambos solvers comparten dos estrategias para mantener el uso de recursos acotado independientemente de la duración o resolución de la simulación:

- **Rolling buffer:** en lugar de almacenar todos los pasos temporales en memoria, se mantienen únicamente dos o tres grillas activas que se rotan mediante reasignación de punteros (`pp, p, cur = p, cur, pp`). El costo de memoria es $O(n^2)$ constante.
- **Submuestreo temporal:** si la simulación requiere más de 1000 pasos temporales, solo se guarda uno de cada $\lfloor n_t/1000 \rfloor$ pasos. Esto limita el tamaño de la respuesta JSON a un máximo de 1000 frames sin afectar la estabilidad numérica, que depende del paso Δt interno y no de cuántos frames se exportan.

6. Aplicación Móvil en Unity

La aplicación Android constituye el cliente interactivo del sistema. Su función es recibir los datos del estudiante, comunicarse con el servidor, transformar las soluciones numéricas en una representación tridimensional y administrar la interacción táctil y sensorial. La resolución por diferencias finitas se ejecuta exclusivamente en Go; el cliente no contiene una segunda implementación del método numérico. Esta separación evita resultados divergentes, reduce la carga del teléfono y permite actualizar los solvers sin recompilar el APK.

El teléfono Android del estudiante cumple la función de IHM del proyecto. Se eligió porque constituye un recurso disponible para la totalidad de los alumnos de la cátedra, lo que permite desarrollar la actividad sin exigir la adquisición ni la disponibilidad de equipamiento adicional. La pantalla táctil recibe la identificación, los parámetros y la selección de nodos, mientras que la unidad de medición inercial (IMU) aporta la orientación utilizada para explorar la escena tridimensional. Esta decisión elimina la necesidad de un periférico externo y facilita el uso de la plataforma en la cátedra de Matemática Avanzada mediante un dispositivo de uso cotidiano. De manera complementaria, la aplicación conserva un modo estereoscópico para Google Cardboard. Este modo no altera el requisito principal: el teléfono por sí solo continúa siendo suficiente para realizar la actividad.

6.1. Entorno de desarrollo y organización por escenas

El proyecto se desarrolló con **Unity 6.4**, versión 6000.4.2f1. Utiliza Universal Render Pipeline 17.4.0, Input System 1.19.0, uGUI 2.0.0, TextMeshPro y Google Cardboard XR Plugin [4, 6]. La dependencia de Cardboard permite ofrecer dos formas de visualización seleccionables durante la ejecución: pantalla plana monoscópica y pantalla estereoscópica para visor.

El ejecutable incluye tres escenas:

1. **MenuScene**: identificación, descubrimiento y registro.
2. **HelloCardboard**: ecuación de onda bidimensional.
3. **HeatPlate**: ecuación de calor bidimensional.

La escena inicial es plana y táctil. Una vez confirmado el registro se carga la onda. Desde cada simulación existe un botón para cambiar de modelo, y la sincronización puede producir el mismo cambio automáticamente.

Los componentes principales se resumen en la Tabla 1.

Tabla 1: Componentes principales de la aplicación Unity.

Componente	Responsabilidad
MenuController	Descubrimiento UDP, registro HTTP y acceso.
ResponsiveMenuUI	Construcción adaptable de la pantalla inicial.
CardboardStartup	Arranque condicional de la cámara plana o del subsistema XR.
DisplayModeController	Selección persistente entre pantalla plana y Google Cardboard.
MobileGyroscopeCamera	Lectura, calibración y suavizado de sensores.
Membrana2D	Petición y reproducción de la onda.
WaveParameterPanel	Parámetros, altura y edición de nodos.
HeatPlate2D	Petición y reproducción del calor.
HeatParameterPanel	Parámetros térmicos y navegación.
SimulationSync	Seguimiento del estado docente.
Coordinator	

Los paneles se construyen durante la ejecución. Esto concentra estilos, anclajes y eventos en componentes C#, y reduce la dependencia de jerarquías serializadas.

6.2. Modos de visualización y sensores

La aplicación inicia el menú como una interfaz plana y, dentro de las escenas de simulación, permite alternar entre dos modos. El modo predeterminado presenta una sola imagen a pantalla completa y utiliza la IMU del teléfono, por lo cual no requiere un visor. El modo VR inicializa Google Cardboard XR, habilita el renderizado estereoscópico de URP y genera una imagen corregida para cada ojo. La selección se conserva al cambiar entre las escenas de onda y calor.

```
1 XRManagerSettings manager = XRGeneralSettings.Instance != null
2   ? XRGeneralSettings.Instance.Manager
3   : null;
4
5 if (DisplayModeController.IsVrModeRequested)
6 {
7     yield return StartCardboard(manager, mainCamera, displayMode);
8 }
9 else
10 {
11     yield return StartFlatMode(manager, mainCamera, displayMode);
12 }
```

Código 28: Selección del modo de visualización.

En Android se prioriza `GameRotationVector`, que estima la orientación sin depender del norte magnético. Si no está disponible se usa `RotationVector`; como último recurso se integran las velocidades angulares del giroscopio [5].

Al ingresar se guarda la rotación inicial y se espera un intervalo breve antes de fijar la referencia. La rotación relativa es

$$q_r = q_0^{-1}q, \quad (18)$$

donde q es la actitud actual y q_0 la referencia. El orden expresa el movimiento en el sistema local inicial y evita intercambiar cabeceo y giro lateral en horizontal. Se conservan estos dos movimientos y se fija el alabeo en cero:

$$q_c = q_i q(\theta_x, \theta_y, 0). \quad (19)$$

La interpolación esférica utiliza

$$\lambda = 1 - e^{-s\Delta t}, \quad (20)$$

Aquí q_i representa la orientación inicial de la cámara en la escena, antes de aplicar el movimiento relativo medido por el teléfono. El orden de multiplicación conserva esa orientación como referencia y aplica luego los ángulos relativos de cabeceo y giro lateral. En la expresión de suavizado, s controla la respuesta del filtro. Así se mantiene la orientación original de la escena sin perder una respuesta natural al movimiento del teléfono.

El seguimiento implementado es rotacional, con tres grados de libertad (3DoF): permite observar la escena al modificar la orientación, pero no determina la posición del teléfono en el espacio como lo haría un sistema de seis grados de libertad (6DoF). El campo de visión y la corrección óptica dependen del perfil del visor configurado por Cardboard, por lo que la aplicación no fija un valor angular único.

En modo VR, el seguimiento de cabeza es provisto por Cardboard mediante el componente de seguimiento de pose de Unity. Los paneles táctiles planos se ocultan para no superponerse a las imágenes estereoscópicas y la cruz nativa de Cardboard permite regresar al modo plano. Esta variante deja preparada la visualización para quienes dispongan de un visor. La conexión de un mando externo y el mapeo de sus botones para recorrer paneles, seleccionar nodos y modificar parámetros se consideran una implementación futura; por lo tanto, el informe no los presenta como una capacidad concluida de la versión actual.

6.3. Menú de inicio adaptable

La pantalla de ingreso funciona en posición horizontal y se adapta a diferentes resoluciones, relaciones de aspecto, esquinas redondeadas y recortes de cámara. La columna izquierda presenta la identidad del simulador y una onda animada; la derecha contiene una tarjeta con nombre, legajo, estado del servidor y conexión.

Fondo, tarjeta, bordes y onda se generan con componentes gráficos de Unity. No dependen de una captura fija y conservan nitidez. El `CanvasScaler` utiliza una referencia horizontal de 1920×1080 y combina ancho y altura:

```
1 scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;  
2 scaler.referenceResolution = new Vector2(1920f, 1080f);  
3 scaler.screenMatchMode =  
4     CanvasScaler.ScreenMatchMode.MatchWidthOrHeight;  
5 scaler.matchWidthOrHeight = 0.5f;
```

Código 29: Escalado adaptable del menú.

La zona segura de Android se transforma en anclajes normalizados:

$$\mathbf{a}_{\min} = \left(\frac{x_{\min}}{W}, \frac{y_{\min}}{H} \right), \quad \mathbf{a}_{\max} = \left(\frac{x_{\max}}{W}, \frac{y_{\max}}{H} \right). \quad (21)$$

Para pantallas panorámicas se mantienen dos columnas equilibradas. En formatos cercanos a 4 : 3, la identidad visual se estrecha y la tarjeta recibe mayor ancho. Los textos usan autoajuste, mientras que campos y botones conservan alturas táctiles.

La IP manual permanece oculta durante el flujo normal. Un enlace la despliega cuando la red bloquea el broadcast. Durante la conexión se deshabilitan los campos, el botón cambia a *CONECTANDO...* y un indicador muestra búsqueda, éxito o error.

6.4. Descubrimiento, registro y comunicación

Sin dirección manual, el cliente transmite `RVPROYECTO_DISCOVER_V1` por broadcast UDP al puerto 47777. La respuesta identifica servicio y puerto HTTP; la IP se obtiene del origen del datagrama. La búsqueda espera 4,5 segundos y requiere una red local común.

Luego se envía `POST /register` con nombre y legajo. Dirección y datos se guardan en `PlayerPrefs`. La onda se carga solamente después de una respuesta satisfactoria.

El descubrimiento y las peticiones HTTP fueron diseñados para una red local de aula. El tráfico no se cifra y `PlayerPrefs` se utiliza como almacenamiento de conveniencia, no como un mecanismo destinado a proteger información sensible. En consecuencia, la operación prevista requiere una red Wi-Fi conocida y controlada.

Las simulaciones se solicitan con `UnityWebRequest` y JSON. El manifiesto Android declara `INTERNET` y `ACCESS_NETWORK_STATE`; además habilita HTTP local sin cifrar. El cliente conserva únicamente el cambio más reciente y espera 5,2 segundos entre cálculos. Ante HTTP 429 reintenta automáticamente.

6.5. Estructura de datos y reproducción

Onda y calor reciben ancho, alto, número de cuadros y un arreglo lineal. Unity verifica

$$N_{valores} = N_x N_y N_t. \quad (22)$$

El cuadro k y nodo i se localizan mediante

$$j = k(N_x N_y) + i. \quad (23)$$

Si las dimensiones no coinciden, la respuesta se rechaza. La secuencia válida se reproduce a 30 cuadros por segundo con un acumulador basado en `Time.deltaTime`.

Cada cuadro actualiza dos texturas: `_MainTex` contiene el color y `_Displacement` la elevación. Separarlas impide que el color científico altere la geometría.

6.6. Escena de la ecuación de onda

`Membrana2D` usa `POST /solve/wave2d`. El panel modifica L , T , c y condición gaussiana, sinusoidal, triangular o modal. Los modos usan índices a y b entre 1 y 5. También se habilitan amortiguamiento y excitación sinusoidal. La frecuencia natural mostrada es

$$f_{ab} = \frac{c}{2L} \sqrt{a^2 + b^2}, \quad (24)$$

```
1 long expected = (long)result.width
2   * result.height * result.frameCount;
3
4 if (result.width <= 0 || result.height <= 0 ||
5     result.frameCount <= 0 ||
6     expected != result.values.LongLength)
7 {
8     return false;
9 }
```

Código 30: Validación de una grilla recibida.

y la excitación enviada es $f_e = m_f f_{ab}$. Esto permite explorar la resonancia.

La escala cromática usa azul para desplazamientos negativos, verde cerca del equilibrio y rojo para positivos. Se mezcla una cuadrícula clara y se dibujan ambas caras para observar la placa desde arriba o abajo.

Cada vértice posee un punto blanco implementado con partículas. El punto muestrea la misma textura en la misma coordenada UV que el material, por lo cual permanece adherido a la superficie durante la evolución.

6.6.1. Condición inicial personalizada

El usuario puede pausar y editar nodos mediante el retículo central y el giro del teléfono. La elevación se transforma en desplazamiento y se extiende a vecinos dentro de un radio de pincel. Al confirmar se envían `initialWidth`, `initialHeight` e `initialValues`. El servidor interpola la grilla y calcula la evolución. Ante una falla, la forma se conserva para reintentar.

El panel y el control de altura pueden ocultarse. Los parámetros se envían después de un breve período sin escritura para evitar una petición por carácter.

6.7. Escena de la ecuación de calor

HeatPlate2D usa `POST /solve/heat2d`. El usuario controla L , T , difusividad α y condición inicial gaussiana, de bordes o de placa.

Las temperaturas se normalizan en $[0, 1]$. La escala recorre oscuro, amarillo, naranja y rojo. También se agrega una grilla y elevación moderada:

$$h = \text{sat}(0,5 + \eta T). \quad (25)$$

El panel permite cambiar la altura y regresar a la onda.

6.8. Sincronización docente

`SimulationSyncCoordinator` persiste mediante `DontDestroyOnLoad` y consulta `GET /sync/status` cada 1,5 segundos. La respuesta indica actividad, ecuación, parámetros y versión incremental.

Si la ecuación publicada no coincide con la escena actual, Unity cambia de escena. Si coincide pero la versión cambió, solicita la nueva solución. El sondeo no ejecuta el solver por sí mismo y las versiones evitan aplicar repetidamente una orden.

```
1 string targetScene = status.ecuacion == "calor2d"  
2   ? HeatScene  
3   : WaveScene;  
4  
5 if (SceneManager.GetActiveScene().name != targetScene)  
6 {  
7     lastAppliedVersion = status.version;  
8     SceneManager.LoadScene(targetScene);  
9     yield break;  
10 }
```

Código 31: Cambio de escena sincronizado.

Una interrupción no detiene la animación recibida: se omite esa consulta y se reintentará en el siguiente intervalo. Así, el profesor propaga un modelo común sin conexiones persistentes.

7. Instalación y Reproducción del Sistema

Este capítulo describe el procedimiento para poner en funcionamiento el servidor, abrir el proyecto de Unity, generar la aplicación Android y reproducir una sesión completa entre profesor y estudiantes. Se considera que el usuario ya dispone en su computadora de las carpetas **Server** y **RealidadVirtual**. El teléfono no resuelve los métodos numéricos, sino que recibe del servidor los resultados que debe representar.

7.1. Requisitos previos

La instalación utiliza los elementos enumerados en la Tabla 2.

Tabla 2: Requisitos para reproducir el sistema.

Elemento	Requisito
Equipo servidor	Windows, Linux o macOS, conectado a la misma red local que los teléfonos.
Go	Versión 1.25.1, de acuerdo con Server/go.mod .
Unity	Unity 6000.4.2f1 con Android Build Support, Android SDK, NDK y OpenJDK.
Teléfono	Android 8.0 (API 26) o superior, con pantalla táctil y sensor de orientación o giroscopio.
Red	Punto de acceso Wi-Fi sin aislamiento entre clientes; TCP 8080 y UDP 47777 habilitados.
Cable USB	Opcional; solo es necesario para instalar mediante ADB o depurar desde Unity.

El teléfono cumple dos funciones: ejecuta el cliente Android y actúa como dispositivo IHM. Se seleccionó porque constituye un recurso disponible para la totalidad de los alumnos de la cátedra, lo que permite desarrollar la actividad con el curso completo sin requerir la adquisición ni el préstamo de equipamiento especializado. La interacción se realiza mediante la pantalla táctil y la IMU. No se requiere visor Cardboard ni controlador externo. De forma opcional puede utilizarse un Google Cardboard para activar la visualización estereoscópica. El manejo integral mediante un mando externo se propone como trabajo futuro y no forma parte de los requisitos para reproducir la versión entregada.

7.2. Preparación y ejecución del servidor

7.2.1. Descarga de dependencias

Desde una terminal ubicada en **Server** se ejecuta:

Código 32: Preparación del servidor Go.

```
go mod download
go build
```

La primera orden descarga Gin y las dependencias declaradas por el módulo. La segunda comprueba que el servidor pueda compilarse en el equipo anfitrión.

7.2.2. Inicio mediante el lanzador gráfico

El modo recomendado se inicia desde la carpeta **Server** con:

Código 33: Inicio mediante el lanzador.

```
go run .
```

El programa abre una página local de configuración. Se conserva el puerto 8080 y se pulsa el botón para iniciar. El lanzador muestra las direcciones IPv4 del equipo; estas sirven para la conexión manual si el descubrimiento automático no está disponible.

Como alternativa, el servidor puede iniciarse directamente, sin lanzador:

Código 34: Inicio directo del servidor.

```
go run . -port 8080
```

Al arrancar se habilita la API HTTP en todas las interfaces de red y el respondedor de descubrimiento en UDP 47777. En Windows, si aparece una consulta del firewall, se debe permitir el acceso al menos para redes privadas. En una configuración manual se crean reglas de entrada para TCP 8080 y UDP 47777.

7.2.3. Verificación del panel docente

En el equipo servidor se abre `http://localhost:8080`. El profesor ingresa el código configurado en `main.go`; luego accede al panel, desde el cual puede ver clientes, calcular soluciones y publicar o detener una sincronización. Para evitar exponer credenciales en una distribución pública, se recomienda cambiar el código antes de desplegar el sistema fuera de la red de laboratorio.

7.3. Apertura y verificación del proyecto Unity

1. Abrir Unity Hub y comprobar que esté instalada la versión 6000.4.2f1 con Android Build Support y sus herramientas.
2. Seleccionar *Add project from disk* y elegir la carpeta `RealidadVirtual`.
3. Esperar la importación de paquetes y la compilación de scripts.
4. Abrir *File > Build Profiles* y seleccionar Android. Si todavía no está activo, utilizar *Switch Platform*.

5. Verificar que las escenas aparezcan habilitadas en este orden:

`MenuScene` → `HelloCardboard` → `HeatPlate`.

El proyecto está configurado para orientación automática, pantalla completa, API mínima 26, permiso de Internet y tráfico HTTP en la red local. El identificador de paquete puede cambiarse en *Player Settings* si el APK debe coexistir con otra compilación del mismo proyecto.

7.4. Prueba previa dentro del editor

Antes de generar el APK es conveniente comprobar el intercambio de datos cliente–servidor:

1. Iniciar el servidor y dejarlo activo.
2. En Unity, abrir `MenuScene` y pulsar *Play*.
3. Desplegar la dirección manual e ingresar `http://127.0.0.1:8080`; el broadcast puede depender de la configuración del editor y no es necesario para esta prueba local.
4. Completar nombre y legajo, y pulsar *Conectar*.
5. Confirmar en el panel docente que el alumno fue registrado y verificar que Unity cargue la escena de onda.
6. Cambiar parámetros de onda y calor. El cliente debe enviar la petición, recibir todos los cuadros calculados y reproducirlos sin ejecutar el solver.

La orientación por IMU no puede evaluarse de manera representativa con el ratón del editor. Esa verificación se completa en el teléfono.

7.5. Generación e instalación del APK

1. En *Build Profiles*, confirmar Android y las tres escenas.
2. En *Edit > Preferences > External Tools*, utilizar las versiones de JDK, SDK y NDK instaladas con Unity para evitar incompatibilidades de Gradle.
3. Pulsar *Build*, elegir una carpeta de salida y guardar el archivo con un nombre identificable, por ejemplo `RV_aplicacion.apk`.
4. Copiar el APK al teléfono y permitir temporalmente la instalación desde esa fuente, o activar depuración USB e instalarlo con ADB:

Código 35: Instalación opcional mediante ADB.

```
adb install -r RV_aplicacion.apk
```

La opción `-r` conserva los datos de una versión anterior. Para una prueba completamente limpia se puede desinstalar previamente la aplicación desde Android.

7.6. Reproducción de una sesión completa

1. Conectar la computadora del profesor y todos los teléfonos a la misma red Wi-Fi.
2. Iniciar el servidor y comprobar el panel docente.
3. Abrir la aplicación en cada teléfono, mantenerlo en horizontal, ingresar nombre y legajo, y pulsar *Conectar*.
4. Esperar la detección automática. Si no responde, desplegar el campo manual e ingresar la IPv4 mostrada por el lanzador, por ejemplo `http://192.168.1.20:8080`. En el teléfono nunca debe utilizarse `localhost`, porque esa dirección identifica al propio dispositivo.
5. Confirmar desde el panel que todos los estudiantes aparecen registrados.
6. En cada cliente, modificar parámetros, pausar o editar nodos y solicitar una nueva solución. Comprobar que la animación, la malla y el color se actualizan.
7. Desde el panel docente, elegir onda o calor, establecer sus constantes y activar la sincronización. En aproximadamente 1,5 segundos, todos los clientes deben cambiar al modelo y versión publicados.
8. Desactivar la sincronización y verificar que cada estudiante recupere el control local de sus parámetros.

Durante la exploración, el usuario gira el teléfono para orientar la cámara y utiliza la pantalla para operar paneles, mover la altura de la placa y seleccionar nodos. Esta combinación materializa la IHM prevista para la actividad de Matemática Avanzada. Si dispone de un Google Cardboard, puede pulsar *MODO VR* antes de colocar el teléfono en el visor. La cruz nativa de Cardboard permite regresar posteriormente a la pantalla plana. En esta modalidad los paneles se ocultan, ya que su operación mediante un mando Bluetooth constituye una ampliación futura.

7.7. Diagnóstico de problemas frecuentes

Tabla 3: Comprobaciones ante fallas de instalación o conexión.

Síntoma	Comprobación y corrección
El servidor no se detecta	Verificar la misma Wi-Fi, desactivar el aislamiento de clientes, habilitar UDP 47777 y probar la IPv4 manual.
No se conecta por IP	Usar la IP de la computadora, no <code>localhost</code> ; comprobar TCP 8080 y que el servidor continúe activo.
HTTP 429	Esperar al menos cinco segundos. El servidor limita la frecuencia de cálculo y el cliente reintenta el último cambio.
Gradle falla al compilar	Seleccionar JDK, SDK y NDK incluidos con Unity; confirmar Android Build Support y espacio libre.
La escena no cambia	Revisar el orden de escenas y confirmar que el registro devolvió una respuesta satisfactoria.
La orientación no responde	Confirmar que el dispositivo posea sensor de rotación o giroscopio, mantenerlo horizontal y reiniciar la escena para recalibrar.
La sincronización no llega	Verificar <code>/sync/status</code> , la conectividad y que el profesor haya activado una nueva versión.

7.8. Lista de verificación final

La reproducción se considera satisfactoria cuando se cumplen simultáneamente los siguientes puntos:

- el panel docente abre y el servidor responde en la red local;
- el teléfono descubre el servidor o se conecta mediante IPv4 manual;
- el registro del alumno aparece en el panel;
- onda y calor se calculan en Go y se reproducen en Unity;
- la pantalla táctil y la IMU permiten operar la IHM en el teléfono;
- los cambios docentes de modelo y parámetros se propagan a los clientes;
- la aplicación permite alternar entre vista plana a pantalla completa y un modo estereoscópico opcional para Google Cardboard;
- la cruz nativa de Cardboard permite regresar al modo plano, mientras que el control completo mediante mando queda documentado como trabajo futuro.

8. Pruebas

Esta sección documenta las pruebas realizadas sobre el sistema. En el lado del servidor se verificó la estabilidad bajo carga concurrente, la correcta aplicación del limitador de tasa por IP, el registro de clientes y la exportación de la planilla de usuarios. En el lado del cliente Unity se comprobó el flujo completo de visualización VR, la sincronización docente y los distintos modos de simulación.

8.1. Servidor

Prueba de carga — clientes concurrentes y limitador de tasa

Para observar el comportamiento del servidor ante solicitudes concurrentes se desarrolló el script `many_clients_test.py`, que simula 40 clientes conectándose de forma simultánea. Cada cliente corre en un hilo independiente (`threading.Thread`): primero se registra en el servidor mediante `POST /register` inyectando la cabecera `X-Forwarded-For` para aparentar IPs distintas y, una vez registrado, entra en un bucle donde selecciona al azar uno de diez conjuntos de parámetros de onda y solicita `GET /solve` con un intervalo aleatorio entre peticiones.

Para ejercitar el limitador de tasa, un subconjunto de clientes se configuró con un intervalo inferior al mínimo permitido por el servidor, de modo que sus solicitudes fueran rechazadas con HTTP 429. En la Figura 2 se observa el resultado: la columna `429` registra los rechazos en los clientes que solicitaron con demasiada frecuencia, mientras que la columna `ERR` permanece en cero para todos, confirmando que el servidor no arrojó ningún error interno bajo carga.

Stress test – http://localhost:8080 clientes: 40 tiempo: 44s req/s: 8.04 OK 250 429 105 ERR 0 Ctrl+C para detener															
#	Nombre	R	OK	429	ERR	Estado	Ec.	#	Nombre	R	OK	429	ERR	Estado	Ec.
001	Ana García	✓	7	2	0	200 OK	onda2d	021	Valentina Daza	✓	6	4	0	200 OK	onda2d
002	Bruno Méndez	✓	8	2	0	200 OK	onda2d	022	Walter Espinoza	✓	6	3	0	429 rate limit	onda2d
003	Carla Ibáñez	✓	6	4	0	200 OK	onda2d	023	Ximena Figueroa	✓	6	2	0	200 OK	onda2d
004	Diego Torres	✓	6	1	0	200 OK	onda2d	024	Yamil Godoy	✓	5	3	0	200 OK	onda2d
005	Elena Ruiz	✓	6	3	0	200 OK	onda2d	025	Zoe Huerta	✓	6	4	0	429 rate limit	onda2d
006	Franco López	✓	5	5	0	429 rate limit	onda2d	026	Agustín Ibarra	✓	6	3	0	429 rate limit	onda2d
007	Gisela Molina	✓	7	2	0	200 OK	onda2d	027	Belén Juárez	✓	7	1	0	200 OK	onda2d
008	Hernán Díaz	✓	6	2	0	200 OK	onda2d	028	Camilo Keller	✓	7	1	0	200 OK	onda2d
009	Ivana Castro	✓	7	3	0	429 rate limit	onda2d	029	Delfina Luna	✓	7	3	0	200 OK	onda2d
010	Julián Romero	✓	6	2	0	200 OK	onda2d	030	Emiliano Mazo	✓	7	2	0	200 OK	onda2d
011	Karen Suárez	✓	6	4	0	429 rate limit	onda2d	031	Florencia Norieg	✓	6	4	0	429 rate limit	onda2d
012	Lucas Fernández	✓	6	2	0	429 rate limit	onda2d	032	Gastón Ortega	✓	6	3	0	200 OK	onda2d
013	Martina Aguirre	✓	6	3	0	429 rate limit	onda2d	033	Hana Ponce	✓	7	2	0	200 OK	onda2d
014	Nicolás Herrera	✓	7	0	0	200 OK	onda2d	034	Ignacio Quiroga	✓	7	2	0	429 rate limit	onda2d
015	Olivia Peralta	✓	5	3	0	429 rate limit	onda2d	035	Jimena Rivas	✓	5	4	0	200 OK	onda2d
016	Pablo Soria	✓	5	4	0	429 rate limit	onda2d	036	Kevin Salinas	✓	5	3	0	200 OK	onda2d
017	Quimey Vera	✓	6	1	0	200 OK	onda2d	037	Luciana Tapia	✓	7	3	0	200 OK	onda2d
018	Rodrigo Acosta	✓	6	4	0	200 OK	onda2d	038	Maximiliano Urqu	✓	6	4	0	429 rate limit	onda2d
019	Sofía Benítez	✓	7	2	0	200 OK	onda2d	039	Natalia Vargas	✓	7	0	0	200 OK	onda2d
020	Tomás Cabrera	✓	7	3	0	200 OK	onda2d	040	Omar Zanetti	✓	6	2	0	429 rate limit	onda2d

Figura 2: Salida del stress test tras 44 s: 40 clientes simultáneos, 250 respuestas exitosas, 105 rechazos por limitación de tasa y 0 errores.

Una vez finalizada la prueba de carga, el panel docente confirmó que los 40 clientes quedaron registrados correctamente en el servidor (Figura 3). El panel muestra para cada

alumno su nombre, legajo, IP simulada y la hora del último request, lo que permite al profesor verificar la actividad en tiempo real.

The screenshot shows a web interface titled "Panel Docente". On the left, there is a list of 40 students, each with their name, ID (legajo), simulated IP, and last access time. The list includes students like Camilo Keller, Hernán Díaz, Pablo Soria, Walter Espinoza, Franco López, Lucas Fernández, Ana García, Quimsey Vera, and Tomás Cabrera. At the bottom of this list is a button labeled "Exportar planilla".

The main area of the interface is divided into two sections: "SINCRONIZACION - FUERZA ESTOS PARÁMETROS A TODOS LOS ALUMNOS" and "VISUALIZACION LOCAL". Both sections have identical controls: a dropdown menu for "Ecuación" (set to "Onda 2D"), input fields for "L (M)" (set to 5), "T (S)" (set to 3), and "C" (set to 1), a dropdown for "Inicial" (set to "Seno"), and buttons for "SINCRONIZAR" and "LIBERAR" in the top section, and "CALCULAR" in the bottom section. There are also checkboxes for "AMORTIGUAMIENTO" and "EXCITACIÓN", both of which are currently unchecked. A large empty box with the text "Ingrese parametros y presione Calcular" is located below the controls in the bottom section. At the very bottom, there are "PLAY" and "STOP" buttons and a progress bar.

Figura 3: Panel docente mostrando los 40 clientes registrados durante la prueba de carga, con sus IPs y últimos accesos.

Desde el mismo panel se exportó la planilla de usuarios en formato Excel (Figura 4), que consolida nombre, legajo, IP y hora de último acceso de cada cliente en una hoja de cálculo que puede utilizarse como apoyo para el seguimiento de la actividad durante la sesión.

Autoguardado ☐ usuarios-s

Archivo Inicio Insertar Dibujar Disposición de página

VISTA PROTEGIDA Tenga cuidado: los archivos de Internet pue

A1 : X ✓ fx Planilla usuarios conectados

	A	B	C	D
1	Planilla usuarios conectados a entorno de simulación [13/08/2026 01:04:33 p. m.]			
2				
3	Generado el 13/08/2026 a las 01:04:33 p. m. — Total: 40 usuario(s)			
4				
5	Nombre	Legajo	IP	Último acceso
6	Emiliano Mazo	100030	192.168.1.30	13:04:29
7	Kevin Salinas	100036	192.168.1.36	13:04:22
8	Luciana Tapia	100037	192.168.1.37	13:04:29
9	Ivana Castro	100009	192.168.1.9	13:04:25
10	Nicolás Herrera	100014	192.168.1.14	13:04:22
11	Bruno Méndez	100002	192.168.1.2	13:04:24
12	Diego Torres	100004	192.168.1.4	13:04:25
13	Elena Ruiz	100005	192.168.1.5	13:04:27
14	Sofía Benítez	100019	192.168.1.19	13:04:28
15	Zoe Huerta	100025	192.168.1.25	13:04:24
16	Jimena Rivas	100035	192.168.1.35	13:04:24
17	Hana Ponce	100033	192.168.1.33	13:04:22
18	Maximiliano Urquiza	100038	192.168.1.38	13:04:25
19	Natalia Vargas	100039	192.168.1.39	13:04:27
20	Martina Aguirre	100013	192.168.1.13	13:04:25
21	Agustín Ibarra	100026	192.168.1.26	13:04:25
22	Gisela Molina	100007	192.168.1.7	13:04:26
23	Olivia Peralta	100015	192.168.1.15	13:04:27
24	Yamil Godoy	100024	192.168.1.24	13:04:25
25	Omar Zanetti	100040	192.168.1.40	13:04:26
26	Valentina Daza	100021	192.168.1.21	13:04:19
27	Camilo Keller	100028	192.168.1.28	13:04:25
28	Hernán Díaz	100008	192.168.1.8	13:04:27
29	Pablo Soria	100016	192.168.1.16	13:04:26
30	Walter Espinoza	100022	192.168.1.22	13:04:24
31	Franco López	100006	192.168.1.6	13:04:24
32	Lucas Fernández	100012	192.168.1.12	13:04:25
33	Ana García	100001	192.168.1.1	13:04:28
34	Quimey Vera	100017	192.168.1.17	13:04:27
35	Tomás Cabrera	100020	192.168.1.20	13:04:29
36	Rodrigo Acosta	100018	192.168.1.18	13:04:27

< > Usuarios +

Listo

Figura 4: Planilla Excel exportada desde el panel docente con los 40 usuarios registrados durante la prueba.

```

1  """
2  Stress test: multiples clientes simulados contra el servidor Go.
3  Requiere: pip install requests rich
4  """
5  import threading, time, random, requests
6  from datetime import datetime
7  from rich.console import Console, Group
8  from rich.table import Table
9  from rich.live import Live
10 from rich.panel import Panel
11 from rich.text import Text
12 from rich import box
13
14 BASE_URL = "http://localhost:8080"
15 NUM_CLIENTS = 40
16 REQUEST_INTERVAL_MIN = 2.0 # segundos minimo entre requests por cliente
17 REQUEST_INTERVAL_MAX = 8.0 # segundos maximo
18
19 NOMBRES = [
20     ("100001", "Ana Garcia"),      ("100002", "Bruno Mendez"),
21     ("100003", "Carla Ibanez"),    ("100004", "Diego Torres"),
22     # ... 36 nombres adicionales con legajos 100005-100040
23 ]
24
25 SOLVE_REQUESTS = [
26     {"ecuacion": "onda2d", "L": 3, "T": 50, "c": 0.5, "inicial": "seno"},
27     {"ecuacion": "onda2d", "L": 5, "T": 60, "c": 1.0, "inicial": "gauss"},
28     {"ecuacion": "onda2d", "L": 4, "T": 75, "c": 1.5, "inicial": "triangular"},
29     {"ecuacion": "onda2d", "L": 8, "T": 55, "c": 0.8, "inicial": "seno"},
30     {"ecuacion": "onda2d", "L": 2, "T": 90, "c": 2.0, "inicial": "gauss"},
31     {"ecuacion": "onda2d", "L": 5, "T": 80, "c": 1.0, "inicial": "triangular"},
32     {"ecuacion": "onda2d", "L": 7, "T": 65, "c": 1.2, "inicial": "seno"},
33     {"ecuacion": "onda2d", "L": 10, "T": 100, "c": 1.2, "inicial": "triangular"},
34     {"ecuacion": "onda2d", "L": 6, "T": 70, "c": 0.9, "inicial": "gauss"},
35     {"ecuacion": "onda2d", "L": 9, "T": 85, "c": 0.7, "inicial": "seno"},
36 ]
37
38
39 class ClientState:
40     def __init__(self, legajo, nombre, client_id):
41         self.legajo = legajo
42         self.nombre = nombre
43         self.simulated_ip = f"192.168.1.{client_id}"
44         self.registered = False
45         self.register_error = None
46         self.requests_ok = 0
47         self.requests_err = 0
48         self.requests_429 = 0
49         self.last_status = "-"
50         self.last_ecuacion = "-"
51         self.lock = threading.Lock() # protege acceso desde el hilo UI

```

Código 36: Configuración y estado de los clientes simulados

```
1 def register(client):
2     try:
3         r = requests.post(
4             f"{BASE_URL}/register",
5             json={"legajo": client.legajo, "nombre": client.nombre},
6             headers={"X-Forwarded-For": client.simulated_ip},
7             timeout=5,
8         )
9         with client.lock:
10             if r.status_code == 200:
11                 client.registered = True
12             else:
13                 client.register_error = f"HTTP {r.status_code}"
14     except Exception as e:
15         with client.lock:
16             client.register_error = str(e)[:30]
17
18
19 def do_solve(client):
20     params = random.choice(SOLVE_REQUESTS)
21     try:
22         r = requests.get(
23             f"{BASE_URL}/solve",
24             params=params,
25             headers={"X-Forwarded-For": client.simulated_ip},
26             timeout=30,
27         )
28         with client.lock:
29             client.last_ecuacion = params["ecuacion"]
30             if r.status_code == 200:
31                 client.requests_ok += 1
32                 client.last_status = "200 OK"
33             elif r.status_code == 429:
34                 client.requests_429 += 1
35                 client.last_status = "429 rate limit"
36             else:
37                 client.requests_err += 1
38                 client.last_status = f"{r.status_code} error"
39     except Exception as e:
40         with client.lock:
41             client.requests_err += 1
42             client.last_status = str(e)[:20]
```

Código 37: Registro y envío de solicitudes durante la prueba de carga

```
1 def client_loop(client, stop_event):
2     register(client)
3     if not client.registered:
4         return
5     while not stop_event.is_set():
6         do_solve(client)
7         wait = random.uniform(REQUEST_INTERVAL_MIN, REQUEST_INTERVAL_MAX)
8         stop_event.wait(wait) # respeta el evento de parada sin busy-wait
9
10
11 def main():
12     clients = [
13         ClientState(leg, nom, i + 1)
14         for i, (leg, nom) in enumerate(NOMBRES[:NUM_CLIENTS])
15     ]
16     stop_event = threading.Event()
17
18     for c in clients:
19         threading.Thread(target=client_loop, args=(c, stop_event),
20                         daemon=True).start()
21
22     start = time.time()
23     try:
24         # Visualizacion en tiempo real con rich.Live (tabla actualizada c/0.5 s)
25         with Live(console=Console(), refresh_per_second=2, screen=True) as live:
26             while True:
27                 live.update(build_layout(clients, time.time() - start))
28                 time.sleep(0.5)
29     except KeyboardInterrupt:
30         pass
31     finally:
32         stop_event.set()
33         elapsed = time.time() - start
34         total_ok = sum(c.requests_ok for c in clients)
35         total_429 = sum(c.requests_429 for c in clients)
36         total_err = sum(c.requests_err for c in clients)
37         print(f"Resumen: {total_ok} OK {total_429} rate-limited "
38               f"{total_err} errores en {elapsed:.1f}s")
39
40 if __name__ == "__main__":
41     main()
```

Código 38: Ejecución concurrente y resumen de la prueba de carga

Pruebas unitarias en Go

El paquete del servidor incluye además un conjunto de pruebas unitarias escritas con el paquete estándar de Go (`testing`), ubicadas en `wave_api_test.go`. Cada prueba verifica un aspecto concreto del comportamiento numérico o de la lógica de validación, sin levantar el servidor HTTP.

- **TestWaveSolverProducesUnityCompatibleGrid**: ejecuta el solver de onda con parámetros mínimos ($L=1$, $T=1$, $c=1$, condición gaussiana) y comprueba que la grilla resultante tenga al menos 16×16 nodos, sea cuadrada (`Width == Height`), y que la longitud del array de valores sea consistente con `Width * Height * FrameCount`.
- **TestWaveSolverAcceptsCustomInitialGrid**: pasa una grilla personalizada de 3×3 con un único punto central elevado y verifica que, tras la interpolación bilineal, el centro de la grilla del solver sea mayor a 0,5 y que los bordes permanezcan fijos en cero (condición de Dirichlet).
- **TestWaveSolverProducesSelectedMode**: solicita el modo "modo:2:1" y verifica la estructura espacial del primer frame: el primer lóbulo (cuadrante inferior izquierdo) debe ser positivo y el segundo (cuadrante superior izquierdo) debe ser negativo, lo que confirma que la condición modal $\sin(2\pi x/L) \sin(\pi y/L)$ fue aplicada correctamente.
- **Rechazo de modos malformados**: la prueba
`TestParseWaveModeRejectsMalformedValues`
comprueba que `parseWaveMode()` rechaza cadenas malformadas ("modo:") y que acota los índices fuera de rango (modo 9:0 debe resolverse a 5:1).
- **TestHeatSolverProducesUnityCompatibleGrid**: ejecuta el solver de calor y verifica las mismas propiedades de compatibilidad con Unity que el test análogo de onda: grilla cuadrada de al menos 10×10 y payload consistente.
- **Cambio de versión de sincronización**: el test
`TestSyncVersionChangesOnSetAndClear`
crea una instancia de `estadoGlobal` e invoca `ActivarSync()` y `DesactivarSync()`, verificando que el campo `version` incremente en cada operación y que los parámetros almacenados coincidan con los enviados.

Las pruebas se ejecutan con `go test ./...` desde el directorio del servidor.

8.2. Cliente

La verificación del cliente Unity se realizó mediante pruebas funcionales manuales en el editor y sobre un teléfono Android real. El modo principal es monoscópico y no requiere auriculares ni visor Cardboard; el dispositivo se utiliza como ventana móvil y sus sensores controlan la orientación de la cámara. También se dispone de un modo VR opcional que activa la división estereoscópica y el seguimiento de Cardboard.

Se comprobaron los siguientes escenarios:

- **Registro e identificación:** el cliente envía correctamente el nombre y el legajo al endpoint `POST /register` y accede a la escena principal tras recibir respuesta exitosa del servidor.
- **Resolución de onda 2D:** se verificó que para distintas condiciones iniciales (gausiana, triangular, modos normales) la visualización tridimensional mostrara el comportamiento físico esperado: propagación, reflexión en bordes y, en el caso de modos, patrones estacionarios estables.
- **Resolución de calor 2D:** se comprobó la difusión desde el estado inicial hacia el equilibrio térmico para las condiciones *gauss*, *placa* y *bordes*.
- **Sincronización docente:** se activó la sincronización desde el panel del profesor y se verificó que el cliente Unity detectara la nueva versión, cambiara entre onda y calor cuando correspondiera y solicitara la solución con los parámetros sincronizados.
- **Modo excitación forzada:** con la fuente activa, se observó que la amplitud de la membrana aumenta cuando la frecuencia de excitación coincide con la frecuencia natural del modo seleccionado (resonancia).
- **Condición inicial personalizada:** el alumno dibuja una forma arbitraria en la interfaz de la aplicación en modo plano, que se envía al servidor como grilla de valores, y la simulación parte de ese estado inicial interpolado.
- **Menú adaptable:** se verificó el ingreso en orientación horizontal, el respeto de la zona segura, el autoajuste de textos y la distribución de la tarjeta en diferentes relaciones de aspecto. También se comprobó la detección automática y el despliegue alternativo de la dirección manual.
- **Cambio de visualización:** se comprobó en el teléfono el paso desde pantalla plana hacia la vista estereoscópica de Google Cardboard y el retorno mediante la cruz nativa. La integración de un mando externo para operar todas las herramientas mientras el teléfono permanece colocado en el Cardboard queda fuera de las pruebas de esta versión.

9. Conclusiones

El proyecto permitió integrar en una única plataforma el cálculo numérico de ecuaciones diferenciales parciales, la comunicación cliente–servidor y la visualización móvil en tres dimensiones. El servidor desarrollado en Go concentra la resolución de los modelos de onda y calor, mientras que la aplicación Android recibe los cuadros calculados y los representa mediante desplazamiento geométrico, color, malla y nodos. Esta separación evita duplicar el método numérico en Unity y mantiene una única implementación para los clientes conectados.

Desde el punto de vista de la interacción, el teléfono cumple simultáneamente el papel de cliente y de dispositivo IHM. Su elección se fundamentó en que se trata de un recurso disponible para la totalidad de los alumnos de la cátedra. El modo plano permite operar los paneles táctiles, modificar parámetros, cambiar de ecuación y editar condiciones iniciales, mientras que la IMU controla la orientación de la cámara. El modo Google Cardboard agrega renderizado estereoscópico y seguimiento rotacional 3DoF, aunque en la versión desarrollada se utiliza principalmente para observación porque los controles táctiles se ocultan durante su ejecución.

La arquitectura también incorporó descubrimiento automático en la red local y sincronización docente. Estas funciones permiten que el estudiante se conecte sin conocer previamente la dirección del servidor y que el profesor publique una misma configuración de onda o calor para los clientes activos. La prueba concurrente documentada registró, durante 44 segundos, 250 respuestas satisfactorias, 105 respuestas HTTP 429 y ninguna respuesta de error interno. Este resultado describe el comportamiento observado bajo la configuración ensayada y no se utiliza para generalizar la capacidad del sistema a otros entornos.

En conjunto, el desarrollo alcanzó una base funcional para explorar EDPs dentro de una actividad de aula. La principal contribución fue articular métodos numéricos, servicios HTTP, sensores móviles y visualización tridimensional en un flujo reproducible. A su vez, el trabajo permitió aplicar conocimientos de programación concurrente, redes, Unity, Android, representación gráfica e integración de subsistemas, propios de la formación en Ingeniería Mecatrónica.

10. Trabajo Futuro

El presente proyecto establece una base funcional para la simulación interactiva de EDPs en realidad virtual, pero abre también varias líneas de trabajo que exceden su alcance actual y que podrían ser abordadas en etapas posteriores.

10.1. Escalabilidad de la plataforma

La arquitectura actual está concebida para un entorno de aula con decenas de usuarios simultáneos conectados a una red local. Escalar la plataforma a un uso más amplio implicaría abordar al menos los siguientes aspectos:

- **Persistencia de sesiones:** el estado de los clientes se mantiene en memoria volátil del servidor. Incorporar una base de datos permitiría sobrevivir reinicios y registrar el historial de simulaciones de cada usuario.
- **Cómputo distribuido:** para dominios grandes o parámetros que requieran grillas de alta resolución, los solvers podrían distribuirse entre múltiples núcleos o nodos mediante goroutines paralelas o una cola de tareas, reduciendo los tiempos de respuesta bajo carga concurrente.
- **Despliegue en la nube:** migrar el servidor a una instancia en la nube eliminaría la restricción de red local, permitiendo que estudiantes accedan a la plataforma desde cualquier ubicación. Esto requeriría además incorporar un sistema de autenticación robusto en reemplazo del esquema de IP actual.
- **Limitación de recursos por usuario:** implementar colas de trabajo y límites de tamaño de grilla por sesión para evitar que simulaciones costosas de un usuario degraden la experiencia del resto.

10.2. Incorporación de nuevos solvers

La arquitectura modular del servidor facilita la incorporación de nuevas ecuaciones diferenciales. Algunas extensiones de interés pedagógico son:

- **Ecuación de Laplace / Poisson:** resolución de problemas electrostáticos y de potencial en estado estacionario mediante métodos iterativos (Jacobi, Gauss-Seidel, SOR). Permitiría visualizar distribuciones de campo eléctrico en dominios con geometrías arbitrarias.
- **Ecuación de onda 1D:** versión simplificada de la membrana vibrante, ideal como paso introductorio antes de los casos 2D. Su baja carga computacional permitiría resoluciones muy finas y tiempos de simulación largos.
- **Navier-Stokes simplificado:** simulación de flujo de fluidos incompresible en 2D (ecuaciones de corriente-vorticidad). Abre la plataforma a cursos de mecánica de fluidos y dinámica de fluidos computacional.
- **Esquemas implícitos:** para la ecuación de calor, un esquema de Crank-Nicolson o totalmente implícito eliminaría la restricción de estabilidad sobre Δt , permitiendo simular escalas temporales mucho más largas sin incrementar el cómputo por paso.

10.3. Mejoras en la experiencia de realidad virtual

Más allá de los solvers, la experiencia inmersiva podría enriquecerse con:

- **Interacción mediante mando en Google Cardboard:** incorporar soporte para un controlador Bluetooth o dispositivo equivalente, definir acciones para desplazarse por los paneles, confirmar opciones, seleccionar nodos y modificar la condición inicial sin retirar el teléfono del visor. La versión actual ya resuelve el cambio de modo, el renderizado estereoscópico y el seguimiento de cabeza, por lo que esta ampliación se concentra en la capa de entrada e interacción.
- Representaciones volumétricas para problemas 3D, aprovechando la profundidad espacial que ofrece la realidad virtual.
- Modo colaborativo en tiempo real, donde múltiples usuarios en el entorno VR puedan modificar condiciones iniciales conjuntamente y observar el efecto de forma sincrónica.

Referencias

- [1] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. Philadelphia, PA, EE. UU.: Society for Industrial and Applied Mathematics, 2007. doi: <https://doi.org/10.1137/1.9780898717839>.
- [2] The Go Authors, “Documentation - The Go Programming Language.” <https://go.dev/doc/>.
- [3] Gin Web Framework, “Documentation.” <https://gin-gonic.com/en/docs/>.
- [4] Unity Technologies, “Unity User Manual 6.0.” <https://docs.unity3d.com/6000.0/Documentation/Manual/>.
- [5] Android Developers, “Position sensors.” https://developer.android.com/develop/sensors-and-location/sensors/sensors_position.
- [6] Google for Developers, “Quickstart for Google Cardboard for Unity.” <https://developers.google.com/cardboard/develop/unity/quickstart>.